Vortex Gaussians: Real-Time Simulation-Driven Fire and Smoke as Native Gaussian-Splatting Content

ZIJUN XIAO, Georgia Institute of Technology, USA



Fig. 1. Real-time, simulation-driven fire and smoke rendered as native Gaussian-splatting content. *Left:* a flame driven by nine layered 2D grid-free Gaussian vortex-particle simulations, in which every vortex element *is* an emissive anisotropic Gaussian (white-hot core → orange → smoke-darkened tips, with embers). *Middle:* the same solver burning inside a *captured* reconstruction—KITCHEN from Mip-NeRF 360 [Barron et al. 2022], in the pre-trained form released by the authors of 3D Gaussian splatting [Kerbl et al. 2023] and loaded here at 729 826 Gaussians with degree-3 spherical harmonics (§5.5). The flame and the scene's splats share one depth order, so the model loader's lift arms and hydraulic linkage occlude the fire, and the shelf and plants behind it are veiled by the plume, per primitive—rather than the fire being composited in front of the scene as a whole (Figure 10). *Right:* the identical particle set re-shaded as smoke (absorption-dominated compositing with warm underlighting). The fire runs in real time in a browser; the middle panel's captured scene adds 272 ms of rasterization on top of it (§7).

A vortex particle and a 3D Gaussian are the same object seen from two disciplines. We build a fire and smoke system on that identity: the splats a rasterizer draws *are* the elements a vortex method integrates, so velocity, its analytic gradient, and temperature are closed-form functionals of the population already on screen—no grid, no field state off the particles, no resampling pass between physics and pixels, and no linear solve of any kind. The consequence is fire that is simulated forward in real time and lives *inside* a Gaussian-splatting reconstruction rather than in front of it: one depth-sorted emission–absorption pass composites flame, smoke, embers and scene splats together, so each occludes the other per primitive. On a benchmark reconstruction whose foreground is foliage, that is a measurable difference and not only an architectural one: 88.1% of the flame's silhouette receives partial transmittance under our composition against 9.1% under the strongest per-pixel depth-tested baseline, handed the threshold that matches our fire energy exactly. Combustion state lives on the scene's own Gaussians—the flame ignites the reconstruction, fire spreads and chars in place, and burning geometry re-seeds the solver.

Fire and smoke are otherwise absent from these scenes: physics-integrated GS handles solids and liquids, GS fire methods reconstruct observed flames rather than new ones, and the one prior system we are aware of that simulates fire composites ray-marched volumes over an opaque backdrop at seconds per frame. Our single-file WebGL2 prototype runs its shipping configuration at ~60 FPS and an 11.6k-Gaussian quality preset at 24 FPS (1280×800, simulation single-threaded in CPU JavaScript). Those rates are the fire alone; drawing it inside a 729 826-Gaussian benchmark reconstruction costs a further 272 ms of rasterization, and we report that cost rather than fold it away.

Because the fields are closed-form, every approximation can be priced against the thing it replaces, inside one prototype and behind one renderer: the Barnes–Hut treecode against the exact Biot–Savart sum ($\leq 1.5\%$ velocity error at the accuracy setting, and faster than the exact sum at every particle count we can ship), grid freedom against a retained vortex-in-cell baseline (2.3× its simulation step at matched counts), the layered decomposition against a true 3D vortex-particle solver built in the same prototype, and the bucketed scene interleave against an order-exact primitive-level merge (out-of-order depth bounded by one bucket width, residual falling as $1/K$). A deterministic-replay harness makes every reported number bit-reproducible from a seed. What each approximation gives up is identified rather than asserted—including where it costs us.

CCS Concepts: • **Computing methodologies** → **Physical simulation**; *Rasterization*; *Image-based rendering*.

Additional Key Words and Phrases: Gaussian splatting, fire, smoke, vortex particle methods, Biot–Savart, Barnes–Hut treecode, real-time simulation, emissive media, volumetric rendering

1 Introduction

Radiance-field capture has made it routine to bring real environments into the computer as 3D Gaussian-splatting (3DGS) scenes that render photorealistically in real time [Kerbl et al. 2023]. A growing body of work now asks how such scenes can be brought to *life*: physics-integrated Gaussian methods animate elastic solids and

Author's Contact Information: Zijun Xiao, zxiao316@gatech.edu, Georgia Institute of Technology, Atlanta, Georgia, USA.

SIGGRAPH '27, TBD
2026.

liquids by attaching continuum solvers to the reconstructed primitives [Feng et al. 2025; Jiang et al. 2024; Xie et al. 2024; Zhong et al. 2024]. Yet the most visually iconic dynamic phenomena—fire and smoke—remain conspicuously missing from this ecosystem. Existing Gaussian representations of flames are *inverse*: they reconstruct an observed fire from video [Nazarenus et al. 2025; Shui et al. 2024] and can only replay it, as do neural volumetric methods for smoke and fluids [Chu et al. 2022; Gao et al. 2025; Yu et al. 2023]. The one system that synthesizes new fire in reconstructed GS scenes, FieryGS [Shen et al. 2026], runs an offline 256^3 Eulerian solver and ray-marches the resulting volume over the GS scene treated as an opaque backdrop, at seconds per frame. We do not claim a like-for-like speedup over it—its 256^3 grid carries three orders of magnitude more degrees of freedom than our slice stack, and we have not run it—but the gap in *operating regime*, offline versus interactive, is what motivates a differently-shaped design.

This gap is not accidental. Gaseous combustion has neither a surface to mesh nor persistent material points to track, so the standard physics-to-GS recipes (bind Gaussians to a deformation field; skin them to simulation particles) do not directly apply, and volumetric emission seems to demand ray marching.

Our starting point is an observation about *representation*: a Lagrangian vortex particle and a 3D Gaussian are the same kind of object. Both are unordered point primitives carrying a position and a handful of per-point attributes; both live in flat GPU buffers; and both are consumed by a sort-and-blend operator (Biot–Savart summation over particles; front-to-back alpha compositing over splats). We exploit this isomorphism literally: **one particle set is simultaneously the simulation state and the render primitive.** Each vortex particle carries circulation Γ, temperature T, and density d; the renderer reads the very same particle as an emissive anisotropic Gaussian whose mean is the particle position, whose color is a blackbody function of T, whose opacity follows d, and whose covariance is evolved by the local velocity gradient. No voxelization, no marching cubes, no simulation-to-renderer resampling of any kind separates physics from pixels. In the spirit of PhysGaussian's "what you see is what you simulate" [Xie et al. 2024], our principle is *what you simulate is what you splat*.

For dynamics we revisit a production idea. Industrial Light & Magic's film fire system [Horvath and Geiger 2009] showed that a stack of independent, camera-facing 2D slice simulations, driven by a coarse directable 3D pass, is perceptually sufficient for even hero-shot fire—but it ran on a GPU farm at ∼20 s per slice per frame, at slice resolutions four orders of magnitude above ours. We do not claim to make that system real-time; we make an interactive-scale instance of its architecture fully Lagrangian and grid-free, which changes it structurally: each slice runs a 2D *Gaussian vortex-particle* solver in which buoyancy injects circulation through the baroclinic term $\beta\,\partial T/\partial x$, evaluated from the particles' own Gaussian temperature mixture; velocity is recovered by Gaussian-regularized Biot–Savart summation under a Barnes–Hut treecode at $O(N_p \log N_p)$, whose truncated top levels serve simultaneously as the coarse directable field that nudges all slices toward one coherent bulk motion. Because the simulation state lives on particles rather than slice rasters, re-orienting the camera-facing stack changes only the *lift* of in-plane coordinates into world space—the slices re-face the viewer at zero re-simulation cost, where raster-state slices must re-simulate per view. The solver state is exactly the render state; there is no Poisson solve, no pressure, and no grid. Because the slices' particles are already Gaussians, the entire stack—flame, smoke, embers, and the static background scene—feeds a single depth-sorted emission–absorption rasterizer with HDR blackbody shading, so the fire correctly occludes and is occluded by the environment per primitive. It also lights it, though that third verb is the weakest of the three: fire-to-scene illumination is a screen-space warm falloff around the flame's image position, not a transport solve, and it is the part of the composition a captured scene shows up most readily.

We validate the system with an interactive prototype implemented as a single-file WebGL2 application whose simulation runs in JavaScript on the CPU, demonstrating that the method is cheap enough for real time even without GPU compute. We further contribute a decoupled per-layer interface that hot-swaps each slice between the physical solver and a procedural curl-noise source at runtime, which we use both as a development methodology and as an ablation instrument.

In summary, our contributions are:

- **An identity between the simulation's degrees of freedom and the rendered primitives, for a gaseous medium.** The particles a rasterizer draws *are* the elements a vortex method integrates: each splat carries a Lamb–Oseen vortex core at its own mean, and every field the solver needs—velocity by Gaussian-regularized Biot–Savart, its analytic gradient, and temperature as a normalized Gaussian mixture—is a closed-form functional of that same population, with no field state, no grid, and no resampling pass anywhere between physics and pixels (§3, §4, §5). The gradient is not merely available but load-bearing: it transports the covariance the rasterizer draws, $\dot{\Sigma} = \mathbf{L}\Sigma + \Sigma\mathbf{L}^\top$, so the shape of each Gaussian is a solver output rather than an artistic one. Gases have neither a rest configuration nor persistent material points, which is precisely what *vorticity*-carrying particles supply—the rest-configuration-free counterpart of covariance-from-deformation for solids [Xie et al. 2024]. Because the fields are closed-form and evaluable at any point, every approximation in the system is *auditable* against the thing it replaces, and every run is bit-reproducible from a seed (§7).
- **A combustion loop between a forward flame simulation and the splats of a captured scene.** Combustion state—heat, fuel, char, glow—lives *directly on the scene's own Gaussians*: the simulated flame ignites the reconstruction, fire spreads splat-to-splat, charring is written into the splats' rendered attributes, and burning geometry re-seeds the solver with new plumes (§5.7). On the public benchmark capture the whole sequence runs with *nothing lit by hand*: the flame lights the table strip it stands in, the front crosses to the object resting on it, and the object's own splats—which receive exactly zero heat from the flame for the first 11.95 s—ignite at $t = 24.40$ s. That delay is an emergent induction time, not a scripted one, and the system contains no delay parameter (§5.7). It runs in real time on a scene we author; on a captured reconstruction the same loop closes, but a 1/60 s substep costs 99–146 ms

instead of milliseconds—six to nine times slower than real time, on two different captures (§5.7). The spread physics itself is conventional [Hädrich et al. 2021; Pirk et al. 2017]; what is new is the substrate and the closed loop—because burning geometry feeds the solver rather than only the renderer, what ignites next is decided by the simulated flow and not only by an adjacency graph. The loop is made possible by compositing fire and scene as one depth-sorted Gaussian population (§5.6), where prior art composites fire *over* GS scenes as an opaque backdrop [Shen et al. 2026]. We price that ordering twice: against an order-exact primitive-level merge, which bounds the bucketed interleave's error at one bucket width, and against the strongest per-pixel depth-tested alternative on two benchmark reconstructions, where it ties on an opaque occluder and separates on a semi-transparent one (§7, Figures 10 and 11).

- **A grid-free solver whose acceleration structure is made of the same primitive, cheap enough to close that loop interactively.** There is no pressure solve and indeed no linear solve of any kind. The Barnes–Hut tree over moment-matched super-Gaussians serves three roles at once: the $O(N_p \log N_p)$ far-field accelerator, the directable coarse field that couples the slices (its merged top levels, plus ground-mirror images supplying the no-penetration boundary), and the low-dimensional handle that direction forces attach to—replacing both the per-slice grids and the coarse grid solver of layered production fire with objects of exactly the type the solver and renderer already speak (§4.2). Because the state is particles, the camera-facing slice stack re-orients at zero re-simulation cost (§4.3). We claim engineering synthesis here rather than a fluids advance—Gaussian vortex blobs, treecodes and SPH estimation are classical—but the composition has a property none of its parts has alone: the accelerator, the director, and the picture are the same object.

We quantify the design space—per-frame cost scaling in slices, particles, and scene splats; treecode vs. exact Biot–Savart summation (accuracy and crossover); a grid-free vs. grid-based A/B of the same slice dynamics against the vortex-in-cell solver we retain as a baseline; slice orientation; covariance dynamics; and, centrally, the layered approximation itself, measured as energy spectra and cross-plane coherence against a matched true-3D Boussinesq reference—through a deterministic-replay harness that makes every number bit-reproducible from a seed (§7).

2 Related Work

2.1 Fire and Smoke Simulation

Visually plausible gaseous simulation in graphics descends from the unconditionally stable semi-Lagrangian advection of Stam [1999], extended with vorticity confinement to fight numerical dissipation [Fedkiw et al. 2001; Steinhoff and Underhill 1994] and specialized to combustion by Nguyen et al. [2002], whose separate fuel/hot-product phases and blackbody radiance remain the canonical physically based fire model. Heat-threshold fire *spread* over solid substrates—fuel, ignition temperatures, charring—is likewise established, on botanical meshes [Pirk et al. 2017] and at wildfire mesoscale [Hädrich et al. 2021]; we adopt this conventional model but carry its state on the splats of a captured scene (§5.7). Real-time grid solvers moved to the GPU early [Crane et al. 2007; Harris 2004].

Vortex methods discretize the vorticity form of the Euler equations on Lagrangian elements, a lineage in graphics that reaches back to hand-tuned vortex fields for film [Yaeger et al. 1986] and 2D vortex-particle gases [Gamito et al. 1995]—particles [Chorin 1973; Cottet and Koumoutsakos 2000; Park and Kim 2005; Selle et al. 2005], filaments [Angelidis and Neyret 2005; Weißmann and Pinkall 2009, 2010], or sheets [Brochu et al. 2012; Pfaff et al. 2012]—which autosatisfy incompressibility in reconstruction, need no pressure projection, and preserve turbulent detail with striking economy. Their classical bottleneck is velocity recovery: naive Biot–Savart summation is $O(N_p^2)$, mitigated either by particle–grid hybrids (vortex-in-cell) [Cottet and Koumoutsakos 2000]—which reintroduce a background grid—or by treecodes and fast multipole methods [Barnes and Hut 1986; Brochu et al. 2012; Greengard and Rokhlin 1987; Lindsay and Krasny 2001], which keep every degree of freedom on the particles. Zhang and Bridson [2014] brought fast summation to graphics vortex flows with a particle-particle particle-mesh split whose far field lives on a mesh; our far field is instead itself a set of Gaussians (§4.2). Vorticity-based solvers continue to advance, most recently through flow-map formulations—Eulerian [Wang et al. 2024] and vortex-particle flow maps [Wang et al. 2025], the latter high-accuracy and offline where we trade accuracy for a real-time rendering-closed loop. We take the treecode branch, precisely because in our system the particles are also the rendered Gaussians: a grid would cap the solver's effective resolution below what the splats can display, and would reintroduce exactly the field state our representation exists to eliminate. Complementary to resolving vorticity transport, up-res and synthetic-turbulence methods graft procedural detail onto a coarse solve [Kim et al. 2008; Pfaff et al. 2010; Schechter and Bridson 2008]; our curl-noise perturbation (§4.1) is the cheapest member of that family, and everything below the solver's resolved scales is appearance, not dynamics. Our Gaussian-smoothed elements are classical [Beale and Majda 1985; Chorin 1973; Leonard 1980], with the Lamb–Oseen core [Lamb 1932; Saffman 1992] chosen because its induced velocity and velocity gradient are closed-form—and because the core profile is, literally, the rendered footprint. We thereby inherit the Lagrangian virtue that matters most here: *the solver's degrees of freedom are already renderable points.* Brochu et al. [2012] argued that simulation cost should scale with rendered detail and named real-time vortex-based fire an open problem; our system is one answer.

Production layered fire. Horvath and Geiger [2009] decompose film fire into a coarse, directable 3D particle pass (a PIC/FLIP variant [Zhu and Bridson 2005]) and a stack of independent, camera-facing 2D Navier–Stokes slice simulations (typically ~2048^2; film examples at 2048×1556 with 32–128 slices), composited near-to-far with Planck blackbody shading. The insight that fine detail perpendicular to the view plane is perceptually invisible justifies 2D refinement. Their slices never exchange simulation state with one another; coherence comes from the shared coarse forcing, splatted with a Gaussian depth kernel spanning ~8 slices. The system is offline (~20 s per slice per frame on a GPU farm), renders slices as textures

composited against holdout mattes, and its stated limitations—fast-moving cameras, flows perpendicular to the view plane, no inter-slice communication—map directly onto the design decisions we revisit. We replace per-slice grid Navier–Stokes with grid-free 2D vortex particles whose Lagrangian state survives slice re-orientation, replace slice textures with Gaussians shared with the environment, and replace the fixed projection kernel with a coarse global *field*—the top levels of the solver's own treecode—whose influence is a continuously tunable velocity blend. Where their two levels are both grids, ours are both Gaussians.

Layered and billboard volume rendering. Half-angle slicing, volumetric billboards, and spherical billboards render volumes as stacks of blended proxies [Decaudin and Neyret 2009; Umenhoffer et al. 2006]; games have long shipped 2.5D flipbook and card-based fire. These techniques require the three ingredients our renderer gets for free from splatting: depth-sorted blending, parallax from depth offsets, and camera-facing orientation.

2.2 Gaussian Splatting

3DGS [Kerbl et al. 2023] represents radiance fields as anisotropic Gaussians rasterized by tile-based, depth-sorted front-to-back alpha blending, following the EWA volume-splatting lineage [Zwicker et al. 2001]; anti-aliased variants [Yu et al. 2024], exact ellipsoid rendering [Mai et al. 2025], particle-scene ray tracing [Moenne-Loccoz et al. 2024], and physically based volumetric-primitive transport [Condor et al. 2025] refine the formulation. Condor et al. [2025] in particular show Gaussian mixtures are a sound basis for *emissive* media under ray tracing; we target the rasterized, real-time end of that trade-off. Dynamic-scene extensions obtain motion from data—per-frame optimization [Luiten et al. 2024] or learned deformation fields [Wu et al. 2024]—and thus replay observed dynamics rather than generate new ones.

Physics-integrated Gaussians animate reconstructed scenes forward in time: PhysGaussian evolves Gaussian kernels under MPM continuum mechanics [Xie et al. 2024], Gaussian Splashing couples position-based fluids with splat rendering [Feng et al. 2025], Spring-Gaus embeds spring-mass systems [Zhong et al. 2024], and VR-GS makes such coupling interactive [Jiang et al. 2024]. Generative variants infer the material model rather than integrate a chosen one—PhysDreamer distills interaction dynamics from video priors [Zhang et al. 2024], OmniPhysGS learns constitutive models spanning solids and (offline, MPM-simulated) liquids [Lin et al. 2025]. All target solids or liquids—media with material points or surfaces—and none address optically thin emissive volumes. Closest in spirit, Gaussian Fluids [Xing et al. 2025] also solves fluids grid-free on Gaussian degrees of freedom—but by the opposite mechanism at the opposite operating point: its Gaussians serve as *basis functions* for a weak-form velocity field, advanced by per-step optimization at 38–228 s per step, with visualization by separate marker and volume passes rather than by the solution Gaussians themselves. We treat Gaussians as explicit strong-form *vortex elements* advanced by classical time integration in real time, closed into a rendering loop where the same Gaussians are the picture. The two systems agree on the premise—Gaussians are a sound fluid discretization—and disagree on everything downstream of it: weak vs. strong form, optimization vs. explicit integration, three to four orders of magnitude in step time, and whether the solution set itself is what is rendered. We read the agreement as independent evidence for the premise, and our contribution as showing that one Gaussian set can hold *both* ends: solution state and render state at once.

Gaussians for fire. FlameGS [Shui et al. 2024] and Gaussians on Fire [Nazarenus et al. 2025] reconstruct emissive flame light fields from images; PINF [Chu et al. 2022], HyFluid [Yu et al. 2023], FluidNexus [Gao et al. 2025], and Vid2Fluid [Zhao et al. 2025] recover smoke or fluid dynamics from video with physics priors, most recently with hybrid physics-optical Gaussian smoke [Zhang et al. 2026]. These are inverse problems. FieryGS [Shen et al. 2026] is the closest prior system and, to our knowledge, the only prior one that *synthesizes* fire in GS scenes: an MLLM assigns combustion properties to a reconstructed scene, a 256^3 Eulerian solver simulates fire, and a ray-marched volume is composited over the GS scene attenuated as an opaque background, at 2.37 s per frame (0.84 s of which is the fire/smoke ray march). Our system differs on every axis (Table 1): Lagrangian rather than Eulerian simulation, per-primitive depth interleaving rather than backdrop compositing, rasterization rather than ray marching, and real-time rather than offline. Two of its capabilities are orthogonal to ours and we do not have them: MLLM-inferred material properties, and fire-to-scene illumination computed by treating voxels above a temperature threshold as volumetric light sources and shading each Gaussian with a diffuse and specular term, where ours is a screen-space warm falloff (§8). Charring it does have, and on the same principle as ours—the scene's own appearance darkens as char accumulates—which makes the *form* of that operator worth a paragraph rather than a footnote (§5.7).

Table 1. Positioning. "GS-native" (render side) means the phenomenon is rendered as Gaussian primitives depth-interleaved with the scene, not composited over it. "Grid-free solver" means no field quantity is ever discretized on a grid: every degree of freedom the solver integrates is a particle—in our case, the very Gaussian the renderer draws.

	gaseous emissive	forward physics	real time	GS-native	grid-free solver	scene composed
Horvath and Geiger [2009]	✓	✓	—	—	—	matte
PhysGaussian [Xie et al. 2024]	—	✓	—	✓	—	✓
G. Splashing [Feng et al. 2025]	—	✓	—	✓	✓	✓
G. Fluids [Xing et al. 2025]	—	✓	—	—	✓	—
Fire recon. [Nazarenus et al. 2025]	✓	—	(render)	✓	n/a	—
VR-GS [Jiang et al. 2024]	—	✓	✓	✓	—	✓
FieryGS [Shen et al. 2026]	✓	✓	—	—	—	backdrop
Ours	✓	✓	✓	✓	✓	✓

3 Overview

Figure 2 summarizes the system. The domain is a stack of N parallel slice layers (default N=9) spanning the fire volume. Once per frame we rebuild each layer's Barnes–Hut tree, merge its top levels into the coarse director field (§4.2), and advance each layer's 2D grid-free Gaussian vortex-particle simulation (§4.1); every layer exposes its particles through the contract

$$\texttt{Layer.step}(\Delta t) \;\rightarrow\; \{\, \mathbf{x}_i \in \mathbb{R}^2,\; T_i,\; d_i,\; \mathbf{u}_i \in \mathbb{R}^2 \,\},$$

i.e., in-plane position, temperature, density, and velocity. The renderer is the *only* consumer of this contract: it lifts each particle into world space using the layer's slice frame (§4.3) and instances one emissive Gaussian per particle (§5). Because the contract is source-agnostic, any layer can be hot-swapped at runtime between the physical solver and a cheap procedural curl-noise plume [Bridson et al. 2007]—a property we exploit for incremental system verification and for ablations (§7).

Scope of the grid-free claim. The claim has six honest edges, stated once here. (i) The 2.5D slice decomposition remains: grid-free refers to how each slice's fields are *represented*, not to the dimensional reduction itself, which is a modeling choice whose price we measure in §7. (ii) The direct buoyancy force is still non-solenoidal (§4.1). (iii) The Barnes–Hut tree, the Morton sort behind it, and the fire-spread spatial hash are *index structures* over the Gaussians—rebuilt from particle state each frame, carrying no independently evolved field; nothing is ever sampled onto them and back. They are the analogue of a renderer's BVH, not of a solver's MAC grid. (iv) Velocity induction uses each particle's isotropic core σ_i, for which the Biot–Savart integral has a closed form; the rendered covariance Σ_i is that core's flow-sheared envelope (§5.2), sharing mean and scale but not orientation. An anisotropic induction kernel has no elementary closed form and is future work. (v) Particles carry two temperatures—a linearly cooled dynamics temperature driving buoyancy and baroclinic injection, and a nonlinearly cooled color temperature driving shading (§4.1)—so the mixture the solver differentiates and the colors on screen are two scalar fields over the *same* means and kernels, not one field. (vi) Circulation Γ_i is solver state the rasterizer never draws directly; it reaches the screen only through the velocities and covariances it induces. The defensible identity is therefore: one population and one set of means, every solver field a closed-form functional of it, and no intermediate field representation anywhere—not that every attribute is shared. The rendered extent is one that is *not*: the vortex core σ_0 is a regularization length inside the kernel, while the splat's on-screen size is a display scale that never enters the solver. The two are coupled only through the overlap condition, and we measure that coupling rather than assert it (§efsec:results). What the solver does drive in the rendered Gaussian is its *shape*: the covariance is transported by the analytic velocity gradient.

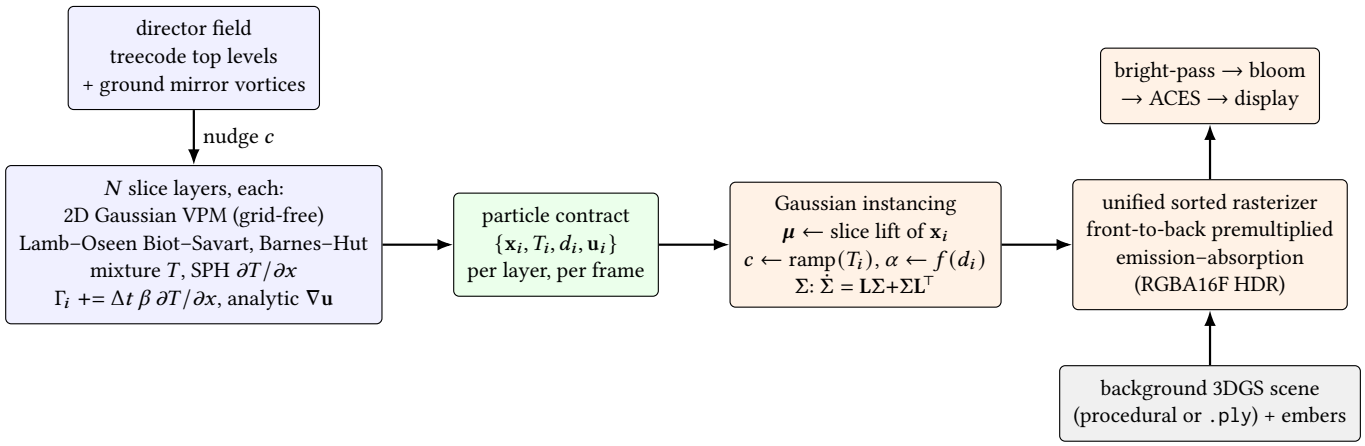


Fig. 2. System overview. A directable coarse field—the top levels of the solver's own Barnes–Hut tree—drives N per-slice 2D grid-free Gaussian vortex-particle simulations (left, §4). Each slice exposes its particles through a minimal contract (center); the renderer lifts each particle *directly* into an emissive anisotropic Gaussian (§5) and rasterizes all Gaussians—fire, smoke, embers, and the background 3DGS scene—in one depth-sorted emission–absorption pass with HDR post-processing. There is no voxelization or resampling anywhere in the pipeline, nor any solver grid: every field the solver evaluates is a closed-form functional of the rendered Gaussians themselves.

4 Two-Level Layered Vortex Simulation

4.1 Per-Slice Gaussian Vortex Dynamics

Each slice simulates buoyant flow in the vorticity–streamfunction form of the incompressible Euler equations. In 2D, vorticity is the scalar $\omega = \partial_x u_y - \partial_y u_x$ and the vortex-stretching term vanishes identically, leaving

$$\frac{\partial \omega}{\partial t} + (\mathbf{u}\cdot\nabla)\,\omega = \big(\nabla \times \mathbf{f}_b\big)_z = \beta\,\frac{\partial T}{\partial x}, \tag{1}$$

where the right-hand side is the *baroclinic* source obtained by taking the curl of Boussinesq buoyancy $\mathbf{f}_b = \beta\,(T - T_{\mathrm{amb}})\,\hat{\mathbf{y}}$: horizontal temperature gradients at the plume edge inject the counter-rotating vortex pairs that roll rising columns into mushroom caps. This single term closes the temperature → buoyancy → vorticity loop that gives fire its characteristic motion [Nguyen et al. 2002; Selle et al. 2005].

Vorticity is carried by the splats themselves. Each particle i—the same object the rasterizer draws—is a Gaussian vortex element [Chorin 1973; Leonard 1980] with circulation Γ_i and isotropic core radius σ_i, so the slice's vorticity field *is* the rendered mixture:

$$\omega(\mathbf{x}) = \sum_j \Gamma_j\, G_{\sigma_j}(\mathbf{x} - \mathbf{x}_j), \qquad G_\sigma(\mathbf{r}) = \frac{1}{2\pi\sigma^2}\, e^{-\|\mathbf{r}\|^2/2\sigma^2}. \tag{2}$$

Every field the solver needs is a closed-form functional of this mixture. Each frame the slice performs four grid-free steps over the 2×3 world-unit domain:

(1) *Temperature.* Each particle carries a *dynamics* temperature T_i, heated in the source band by T_i += $\Delta t\, s_0 \exp(-\kappa_s \tilde{x}_i^2)$ and cooled linearly by T_i *= $(1 - \frac{1}{2}\kappa_c \Delta t)$; advection is free, because the carrier advects (the semi-Lagrangian grid step of Eulerian formulations disappears). The temperature *field* is the Shepard-normalized Gaussian mixture

$$\widetilde{T}(\mathbf{x}) = \frac{\sum_j T_j\, w_j(\mathbf{x})}{\sum_j w_j(\mathbf{x})}, \qquad w_j(\mathbf{x}) = e^{-\|\mathbf{x}-\mathbf{x}_j\|^2/4\bar{\sigma}_j^2}, \tag{3}$$

with kernel width $\sqrt{2}\,\bar{\sigma}$ (the geometric mean core of the interacting pair), exact for constant fields and degrading gracefully to $\widetilde{T} \to T_i$ for isolated particles.

(2) *Baroclinic injection.* Each particle accumulates circulation from the local horizontal gradient of the mixture:

$$\Gamma_i \leftarrow (1 - \lambda_\Gamma \Delta t)\,\Gamma_i + \Delta t\,\beta\,\frac{\partial \widetilde{T}}{\partial x}\Big|_{\mathbf{x}_i}, \tag{4}$$

where the gradient is the moment-renormalized SPH difference estimate [Monaghan 1992]

$$\frac{\partial \widetilde{T}}{\partial x}\Big|_{\mathbf{x}_i} = \frac{\sum_j (T_j - T_i)\,(x_j - x_i)\,w_{ij}}{\sum_j (x_j - x_i)^2\, w_{ij}}, \tag{5}$$

which is exact for linear fields under *any* particle distribution—the property that matters at ragged plume edges, where the baroclinic source lives.

(3) *Velocity and its gradient, in closed form.* The mixture (Eq. 2) induces the Lamb–Oseen velocity [Lamb 1932; Saffman 1992]

$$\mathbf{u}(\mathbf{x}) = \sum_j \frac{\Gamma_j}{2\pi}\,\frac{(-r_y,\, r_x)}{\|\mathbf{r}_j\|^2}\Big(1 - e^{-\|\mathbf{r}_j\|^2/2\sigma_j^2}\Big), \qquad \mathbf{r}_j = \mathbf{x} - \mathbf{x}_j, \tag{6}$$

with three properties we use constantly: it is regular at $\mathbf{r}$=0 (self-induction vanishes; particle advection needs no desingularization parameter beyond σ itself); it is solenoidal *analytically, term by term* (each summand is the perpendicular gradient of a streamfunction, so incompressibility of the recovered field holds exactly, with no iterative solve to truncate); and beyond a few σ it reduces to the point-vortex kernel, so the exponential is only ever evaluated in the near field. Its

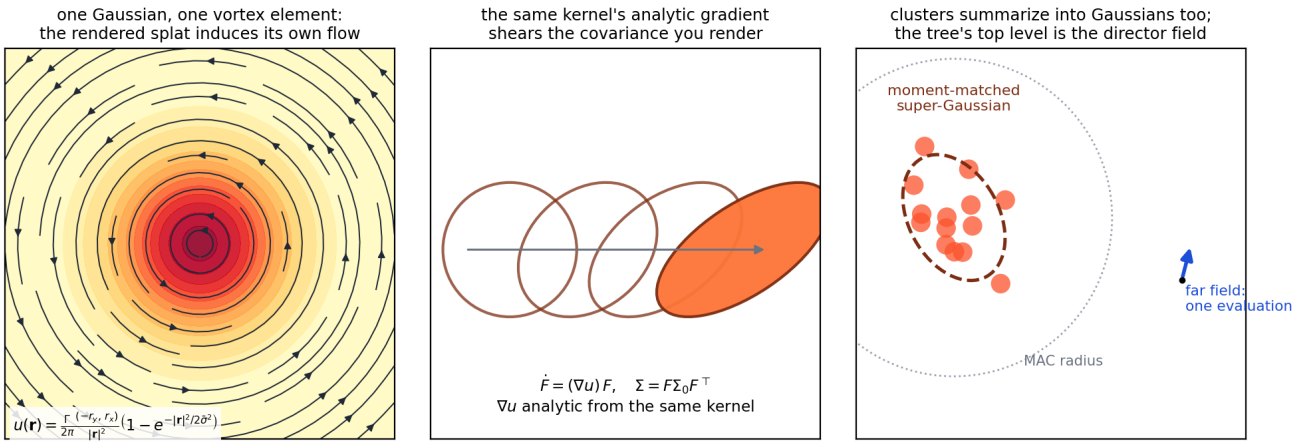


Fig. 3. The identity that organizes the solver: one Gaussian, three closed-form roles. *Left:* the rendered splat *is* a Lamb–Oseen vortex element—the emissive footprint and the induced velocity field are one object, related in closed form (Eq. 6). *Middle:* the same kernel's analytic gradient drives the deformation of the covariance that is rendered (§5.2); no finite differencing anywhere. *Right:* treecode clusters are summarized by moment-matched super-Gaussians, so the far field is also a Gaussian evaluation—and the tree's top level doubles as the director field (§4.2).

spatial gradient $\mathbf{L} = \nabla\mathbf{u}$ is likewise closed-form and is accumulated in the same summation at marginal cost; it drives both the rendered covariance (§5.2)—replacing finite differences with the exact derivative of the field the particles themselves induce—and, in the 3D reference solver, vortex stretching (§7). The sum is accelerated by a Barnes–Hut treecode (§4.2); an exact $O(N_p^2)$ evaluation of Eq. 6 is retained as a reference path and used to measure treecode error (§7).

(4) *Advection.* Each particle adds direct buoyancy $\beta_b \widetilde{T}(\mathbf{x}_i)\,\hat{\mathbf{y}}$, a small divergence-free curl-noise perturbation [Bridson et al. 2007], wind, and the director nudge of §4.2, then advects with explicit Euler.

Note that buoyancy appears twice by design, with independent gains: the baroclinic term (Eq. 4, gain β) generates the *rotational* response—edge rolling and mushroom caps—while the direct term (gain β_b) is an additional control force for bulk rise. Decoupling the two is an artistic choice, not a derivation, and it has a consequence we state plainly: $\nabla\cdot(\beta_b T\hat{\mathbf{y}}) = \beta_b\,\partial T/\partial y \neq 0$ in a thermal plume, so while the recovered field of Eq. 6 is solenoidal term by term, the field that actually advects particles is not. With β_b=0.9 against β=1.4 this term is the same order as the baroclinic response. We therefore claim that the solver needs no pressure solve—indeed no linear solve of any kind—and not that the advecting velocity is divergence-free.

Viscosity and core evolution (optional). Because each element is a Lamb–Oseen vortex, viscous diffusion has an exact per-particle form: core spreading, $\mathrm{d}\sigma_i^2/\mathrm{d}t = 2\nu$ [Leonard 1980]. We expose it as an artistic softening control, default off. We use it as a controlled dissipation knob, not as a convergent viscous solver: pure core spreading famously approximates the wrong equation in the large [Greengard 1985], and the splitting schemes that repair it [Rossi 1996] are unnecessary at our 2–4 s particle lifetimes, over which cores grow by at most a factor of two before recycling.

Thermodynamics and lifecycle. Alongside the dynamics temperature, particles carry a rendered *color* temperature, cooled nonlinearly, $\mathrm{d}T_i^c/\mathrm{d}t = -\kappa\,(T_i^c)^2$, which concentrates brightness at the hot base and produces the natural fast fade toward the flame tips; density decays exponentially. The two-temperature split mirrors the structure of grid-based formulations (a linearly cooled advected field driving dynamics; a nonlinearly cooled particle temperature driving shading)—here both live on the particles. Particles are recycled when cold, exhausted, or out of bounds, and re-emitted at the source (emission balanced against the cull rate, up to a 1300-particle cap), so the particle count—and hence the Gaussian count—stays bounded.

4.2 Treecode Far Field and the Director Field

Barnes–Hut over super-Gaussians. Evaluating Eq. 6 exactly is $O(N_p^2)$. We accelerate it with a Barnes–Hut quadtree [Barnes and Hut 1986], rebuilt from scratch each frame (Morton-sorted, so leaves are contiguous in memory; construction is a fraction of a millisecond and determinism is trivial). Each node aggregates its subtree into a moment-matched *super-Gaussian*: signed total circulation, an $|\Gamma|$-weighted centroid (with a small uniform floor that keeps centroids defined when signed circulations cancel), a signed dipole moment that restores the leading effect of such cancellation—our emission spectrum is near zero-mean in Γ, so monopoles alone systematically underestimate the far field—and a matched core $\sigma_c^2 = \sum_j|\Gamma_j|(\sigma_j^2 + \|\mathbf{x}_j - \mathbf{x}_c\|^2)/\sum_j|\Gamma_j|$ that absorbs the subtree's spatial dispersion. A far node is thus itself a Gaussian vortex element and is evaluated by the same closed form (Eq. 6); the multipole acceptance criterion $\|\mathbf{x}-\mathbf{x}_c\| > \max(s/\theta,\ \frac{3}{2}\sigma_c)$ (node size s, opening angle θ) additionally keeps clusters outside their own cores, where the single-Gaussian surrogate is valid. We report measured relative L_2 velocity error against the exact sum across θ rather than an analytic bound (§7)—the empirical practice of vortex treecodes [Lindsay and Krasny 2001]; at the default θ=0.7 the error is ∼1.5%, below any

difference we could discern in informal A/B flips (we have not run a formal perceptual test; §8).

The director field. Independent slices produce N *uncorrelated* fires. Production layered fire solved this with a splat kernel spanning neighboring slices [Horvath and Geiger 2009]; we instead couple on the simulation side, and the coupler falls out of the data structure we already have. The top levels of the per-layer trees, merged across all slices into ≤16 global super-Gaussians—plus their images reflected through the ground plane with negated circulation, which restore the no-penetration floor without any boundary grid—define a global velocity field $\mathbf{U}$ (evaluated monopole-only, averaged over layers, plus a bulk-buoyancy term from the clusters' mixture temperature). $\mathbf{U}$ represents the directable bulk motion of the whole fire; every particle is nudged toward it:

$$\mathbf{u}_i \leftarrow \mathbf{u}_i + c\,\big(\mathbf{U}(\mathbf{x}_i) - \mathbf{u}_i\big), \qquad c \in [0,1]. \tag{7}$$

c=0 leaves slices fully independent (each layer visibly "its own fire"); c=1 slaves all slices to the shared field (one coherent but overly smooth column); the default c=0.5 retains per-slice turbulent detail on top of a shared bulk motion (Figure 13). This is the two-level decomposition of Horvath and Geiger [2009]—coarse directable motion plus per-slice refinement—with two upgrades: the coupling is a first-class, continuously tunable simulation parameter rather than a fixed projection kernel, and the director level is no longer a grid solver but a handful of super-Gaussians, each an editable vortex element (position, circulation, core size) of exactly the same type the solver and renderer already speak. Both levels of the hierarchy are Gaussians; one tree yields the near-field detail, the far-field acceleration, and the global handle. Direction forces *can* be confined to this low-dimensional level; our prototype currently applies wind and source motion per-particle as well.

Inter-slice vorticity coupling. Equation 7 couples every slice to a shared *global* field, but it does not couple slices to their immediate *neighbors*: two adjacent slices can follow the same bulk motion while their turbulent detail stays uncorrelated, and stacked-2D dynamics structurally drop the out-of-plane transport $\partial^2\omega/\partial z^2$ that would tie them together. We reintroduce this dropped coupling directly, particle to particle. Pairs (i, j) of particles in adjacent slices within kernel range exchange circulation through a symmetric Gaussian weight,

$$\Gamma_i \mathrel{+}= \delta_{ij}, \quad \Gamma_j \mathrel{-}= \delta_{ij}, \qquad \delta_{ij} = \Delta t\,\kappa_z\,\eta_{ij}\,(\Gamma_j - \Gamma_i), \tag{8}$$

with $\eta_{ij} \propto G_{\bar{\sigma}_{ij}}(\mathbf{x}_i - \mathbf{x}_j)$—a particle-strength-exchange discretization [Degond and Mas-Gallic 1989] of the transport the stacking deletes. Conservation of total circulation is now pairwise and *exact by antisymmetry* (each pair contributes $\pm\delta_{ij}$), and the grid form's Neumann ends reduce to the mere absence of a neighbor slice. Production layered fire [Horvath and Geiger 2009] has no slice-to-slice communication at all—its slices are coupled only through the one-way coarse broadcast—so this term fills an obvious hole, but we are explicit about how modest it is: it is the deleted term's three-point structure applied explicitly against the previous frame's circulations, and it carries no $1/\Delta z^2$, so κ_z is a per-frame exchange rate rather than a diffusivity and is not comparable across slice counts. It is *off* in the shipping look (Table 3), a visual preference rather than a verdict: under the spectral protocol of §7 the particle form of the exchange raises adjacent-slice velocity correlation from 0.18 to 0.62 (0.76 combined with coarse coupling, against 0.80 for the matched 3D control; four-seed means), at the price of a measurably smoothed in-plane spectrum—it is a diffusion, and it behaves like one. The grid form of the same operator had measured inconclusively on a snapshot probe, a negative result we retain in §7 as a methodological caution. The two mechanisms are complementary by design—Eq. 7 imposes global bulk coherence, Eq. 8 exchanges local neighbor detail—and must be read against both axes of §7. Crucially, Eq. 8 models the *coupling* that stacking drops; it does *not* recover true 3D vortex stretching $(\boldsymbol{\omega}\cdot\nabla)\mathbf{u}$, whose spectral consequences we treat honestly in §8.

4.3 Slice Geometry

Layer $k \in \{0, \ldots, N-1\}$ is assigned the depth offset $z_k = (k/(N-1) - \frac{1}{2})\,D$ along the stack axis, with a mild per-layer opacity fade $1 - 0.3\,|z_k|$. We support two slice frames:

View-aligned (default). The in-plane axes are the camera's horizontal right vector and the world up vector, and layers are offset along the horizontal camera-forward direction. Slices re-orient continuously as the camera orbits, so the stack always presents its face—the choice of Horvath and Geiger [2009], without their per-view re-simulation, because our simulation state lives on particles rather than on slice rasters.

World-fixed. Slices are pinned to a world plane and offset along the world normal. Orbiting reveals the stack edge-on and the volume collapses into stripes (Figure 12).

Viewed edge-on, a world-fixed stack degrades into visible stripes while the view-aligned stack continues to read as a volume (§7). Because re-orientation only changes the *lift* of in-plane particle coordinates into world space—not the simulation state—view-aligned slicing costs nothing and introduces no re-simulation popping; the known failure modes of view-locked slices under fast camera motion [Horvath and Geiger 2009] are correspondingly milder but not eliminated (§8).

5 Emissive Gaussian Rendering

5.1 Particle-to-Gaussian Mapping

Table 2 lists the mapping. Each particle is instanced as a camera-facing quad whose vertex shader expands the unit square along the Gaussian's major and minor semi-axes and projects with the full perspective camera; the fragment shader evaluates the Gaussian footprint $g = \exp(-3r^2)$ modulated by a four-octave value-noise FBM detail factor that breaks the characteristic "blobby" look of isolated splats. Instance data is a flat 14-float record (position, two axes, fade, T, d, seed, type)—the direct render-side realization of the simulation contract.

5.2 Flow-Driven Covariance

Isotropic splats read as bubbles; real flames are sheared into streaks and wisps by the flow. Analogously to how PhysGaussian transforms covariances by the deformation gradient [Xie et al. 2024], we evolve each particle's in-plane 2×2 covariance directly by the local velocity gradient $\mathbf{L} = \nabla\mathbf{u}$—evaluated analytically from the induction kernel

Table 2. The complete simulation-to-rendering mapping. Every attribute of the rendered Gaussian is read directly off the simulation particle; there is no intermediate representation.

particle attribute	Gaussian attribute	mechanism
position $\mathbf{x}_i$ + layer k	mean $\boldsymbol{\mu}_i$	slice-frame lift (§4.3)
temperature T_i	color c_i	blackbody ramp (§5.4)
density d_i	opacity α_i	§5.3
velocity gradient $\mathbf{L}_i$	covariance Σ_i	Eq. 9
trajectory history	filament axis	Eq. 10

(§4.1), so the shear that streaks a splat is the exact derivative of the field the splats themselves induce, with no finite-difference truncation and no grid Nyquist limit:

$$\dot{\Sigma} = \mathbf{L}\,\Sigma + \Sigma\,\mathbf{L}^{\top}, \tag{9}$$

integrated explicitly with gain k_Σ, relaxed toward identity at rate k_r, and stabilized in closed form: after a 2×2 eigen-decomposition we renormalize $\det\Sigma = 1$ and clamp the leading eigenvalue $\lambda_0 \le \kappa_{\max}$ (with $\lambda_1 = 1/\lambda_0$ from the determinant constraint, this bounds the splat's semi-axis ratio a/b at $\kappa_{\max}$). Under $\operatorname{tr}\mathbf{L} = 0$ exact integration would preserve $\det\Sigma$; the renormalization removes explicit-integration drift. Equation 9 is the differential form of PhysGaussian's $\Sigma = \mathbf{F}\Sigma_0\mathbf{F}^{\top}$ [Xie et al. 2024]—but for a gas, $\mathbf{F}$ and Σ_0 are ill-defined absent a rest configuration, which makes the incremental form the natural one; and because the clamps act on the rendered quantity itself, the unbounded-elongation ("sliver") failure mode reported for Gaussians under sustained shear [Feng et al. 2025; Xie et al. 2024] cannot occur. The splat's semi-axes are then $a = s\sqrt{\lambda_0}$, $b = s\sqrt{\lambda_1}$ along the eigenvectors, so vortical shear visibly stretches blobs into licks of flame (Figure 14). Attaching per-particle anisotropy to a coarse solve echoes anisotropic turbulence particles [Pfaff et al. 2010], with the anisotropy here doubling as the render primitive's footprint.

As an optional third tier, a trajectory *filament* stretch elongates each Gaussian along its recent path using an exponentially lagged shadow position $\mathbf{p}^{\text{lag}}$:

$$\mathbf{p}^{\text{lag}} \leftarrow \mathbf{p}^{\text{lag}} + \eta\,(\mathbf{p} - \mathbf{p}^{\text{lag}}), \qquad a = s + k_f\,\|\mathbf{p} - \mathbf{p}^{\text{lag}}\|, \tag{10}$$

turning the particle set into short path-integrated streaks reminiscent of vortex filaments [Angelidis and Neyret 2005; Weißmann and Pinkall 2010]—an instantaneous-shear complement particularly effective for embers and fast plumes.

5.3 Unified Sorted Compositing

All dynamic and procedural-scene Gaussians—flame, smoke, embers, and the 640-splat background environment—are gathered into one buffer, sorted front-to-back by view depth, and rasterized in a single instanced draw into an RGBA16F target cleared to zero coverage. Fragments output premultiplied $(c_i\alpha_i,\ \alpha_i)$ and are blended with

$$C \leftarrow C + (1 - A)\,c_i\alpha_i, \qquad A \leftarrow A + (1 - A)\,\alpha_i, \tag{11}$$

(GL blend `ONE_MINUS_DST_ALPHA`, `ONE`)—exactly the front-to-back emission–absorption compositing of the 3DGS forward pass [Kerbl et al. 2023; Zwicker et al. 2001]. Opacity is the product of a *footprint* and a per-primitive weight,

$$\alpha_i(r) \;=\; \hat{g}(r)\ \operatorname{clamp}\big((\alpha_0 + \alpha_1 d_i)\,T_i\,\phi_k,\ 0,\ \alpha_{\max}\big), \tag{12}$$

where r is the radial coordinate of the fragment in the primitive's own frame, ϕ_k is a per-layer depth fade, and $\hat{g}(r) = \big(g(r) - g(r_{\max})\big)/\big(1 - g(r_{\max})\big)$ with $g(r) = e^{-\kappa r^2}$ is the Gaussian footprint, pedestal-subtracted and renormalized so that it falls *smoothly* to exactly zero on the quad's inscribed circle ($r_{\max} = 1$), outside of which fragments are discarded. Subtracting the pedestal at the circle rather than at the quad corner matters: a corner-normalized footprint retains a ~5% alpha step along the quad's straight edges, which reads as a visible square outline on large splats—the eye is acutely sensitive to straight-line discontinuities. The *same* $\hat{g}$ is evaluated by flame, smoke, ember and scene fragments; only the weight differs. Fire is optically thin and emission-dominated, so its Gaussians carry low $\alpha_{\max}$, accumulating radiance while barely occluding; smoke is absorption-dominated with high α and a density-modulated gray albedo warmed by residual temperature. The distinction matters: with a constant per-fragment α and the footprint demoted to a binary mask, the primitive is a disc sprite rather than a Gaussian, and the quad's corners become visible wherever the sprite is large on screen. Since emission and absorption are independent per-Gaussian channels, the *same* particle state spans the fire-to-smoke range under two scalar knobs (emission gain, absorption albedo) with no change to geometry, sort, or solver—consistent with Kirchhoff's law, hot particles emit and cooled ones absorb—so a single plume can transition from emissive flame at its hot base to absorptive smoke at its cooled tips. Because occlusion is resolved per primitive by the shared sort, the fire correctly sits *inside* the scene: environment splats in front of the flame occlude it, and the flame occludes those behind—in contrast to compositing a ray-marched volume over the scene as a whole [Shen et al. 2026]. Figure 10 (top left) shows both directions on the capture: the model loader's lift arms and hydraulic linkage stand in front of the flame and cut its white-hot base away, while the shelf, pots and plants behind it are veiled by the plume. The occluder there is opaque plastic, so the first direction is very nearly binary—measured over the flame's silhouette, 12.0% of it is fully blocked and only 2.5% partially transmitted—which matters for what that figure can and cannot settle (§5.6). Figure 11 repeats the comparison on a capture whose foreground is foliage rather than plastic, where 93.9% of the same silhouette is partially transmitting and the two compositions separate. Procedural environment splats share the sorted buffer directly; large *loaded* reconstructions are merged into the same global depth order by bucketed interleaving (§5.6, Figure 10).

5.4 Blackbody HDR Shading and Post-Processing

Fire emission follows incandescence: we map particle temperature through a hand-tuned five-stop ramp approximating Planck blackbody colors [Horvath and Geiger 2009; Nguyen et al. 2002; Pegoraro and Parker 2006] (dark ember red → red → orange → yellow → near-white), scaled by an HDR emissive gain and modulated three ways: a global low-frequency flicker, a per-splat phase jitter (±8%, seed-keyed), and a fine-scale FBM "lick" factor advected upward through the footprint, which breaks the smooth core into tongues. At the cold end the shading crosses over: below $T \approx 0.3$

each splat fades from emission toward an absorbing, ember-tinted soot gray with higher opacity—the crown of a fire is smoke, not dim flame—and its rendered radius grows by up to 1.5× as T falls, the render-side analogue of thermal expansion (both are pure shading terms; the solver state is untouched). Radiance accumulates far beyond display range, so the composited scene passes through a standard HDR chain: bright-pass extraction, five iterations of half-resolution separable Gaussian blur (bloom), then ACES filmic tone mapping [Narkowicz 2016], gamma, vignette, and a 1/255 dither. Combined with the nonlinear particle cooling of §4.1, this reproduces the canonical luminance structure of real flames—saturated white-hot core, graded orange body, smoke-darkened tips—without any texture assets (Figure 16).

5.5 The Captured Scene, and What We Do to It

Four scenes appear in this paper and they have very different standing. We separate them because a claim about compositing into real reconstructions is only worth as much as the reader's ability to check it, and the four sit at different points on that scale.

The first is a synthetic `.ply` of 45 808 splats—a plank floor, a stone wall, a fire ring, fuel logs and a rear post—which we generate (the generator ships in the supplemental). It is not a reconstruction and we do not present it as one. It exists so that every scene-side result in this paper has a version a reader can regenerate byte-for-byte from a seed.

The second is a public benchmark reconstruction: KITCHEN from Mip-NeRF 360 [Barron et al. 2022], in the pre-trained form released by the authors of 3D Gaussian splatting [Kerbl et al. 2023]— 1 852 335 Gaussians at degree-3 spherical harmonics. We did not train it, we cannot influence it, and, unlike the capture below, a reader can download the identical file and repeat every step we describe. It carries the properties that make this problem interesting: a cluttered table of small objects at many depths, a model construction loader whose lift arms, hydraulic linkage and track links are thin foreground geometry, and a trained radiance field whose view-dependent colour was never meant to sit behind an emissive volume. What it does *not* offer is a semi-transparent foreground: the potted plants sit on a shelf behind the table, and from every viewpoint inside the capture's own camera distribution they are behind the tabletop, so nothing translucent can be placed between the camera and a fire standing on the table. That constraint shapes what the occlusion comparison on *this* scene can settle (Figure 10), and it is the reason the next scene is in the paper at all. KITCHEN is the scene of the teaser and of the opaque-occluder comparison.

The third is a second public benchmark reconstruction, in the paper for exactly what the first one lacks: TREEHILL, from the same Mip-NeRF 360 release and the same pre-trained checkpoints [Barron et al. 2022; Kerbl et al. 2023]—3 783 761 Gaussians at degree-3 spherical harmonics. It is an outdoor park capture, and what stands between a camera and the ground behind its low wall is a clipped hedge under a plane tree: leaves, twigs and grass blades, which the reconstruction represents not as a surface but as a lace of low-opacity Gaussians with air visible between them. That is a genuinely semi-transparent foreground. We did not choose it by eye. Across all eight pre-trained Mip-NeRF 360 scenes (and the campsite below besides) we scored candidate fire placements at the releases' own registered cameras on the fraction of the flame's silhouette whose transmittance through the intervening reconstruction falls strictly between 0.05 and 0.95—the fraction that no per-pixel depth test can express—and ranked on it. TREEHILL at camera `_DSC8979` is what we kept: measured on the prepared asset at the shipping camera, 93.9% of the flame's silhouette is partially transmitting at mean transmittance 0.713, against 2.5% partial on KITCHEN. That is a different regime rather than a different degree, and it is the whole reason the scene exists here. The same frame also contains a hard opaque occluder—the plane tree's trunk, immediately left of the flame—so one image carries both cases. It is the scene of Figure 11 and of nothing else.

The fourth is a real trained reconstruction of a campsite: a parked vehicle with camping gear in its open tailgate, a second vehicle, dirt ground and forested hills, captured on an eight-camera 360° rig over ∼4 430 frames with people culled, and trained to 2 998 953 Gaussians with degree-3 spherical harmonics by a commercial pipeline. We neither captured nor trained it; it reached us from a colleague, which is the point—it is a reconstruction produced without any knowledge of this system, in a format we did not choose. It is also, for the same reason, not ours to redistribute.

Loading any capture needs one offline step, and we state it because it discards reconstruction content: an opacity floor, a crop to the region of interest, a seeded uniform subsample to a splat budget, and—where the renderer cannot evaluate what the file carries—a truncation of the spherical harmonics. The three prepared assets sit at very different points on that step. For KITCHEN it is a *filter*, not a subsample: at a floor of 0.35, with the crop box set to $x \in [-12.5, 7]$, $y \in [-5, 8]$, $z \in [-6, 9.5]$ in the file's own frame, 729 826 of the released 1 852 335 Gaussians clear the opacity floor inside it, which is below the 800k budget, so the uniform draw never binds and not one splat is discarded by it. What we render is therefore every Gaussian above the floor inside the crop, at degree 3—the degree it was trained at—and the appearance model is intact. The reduction is 2.5×, and all of it is the floor and the crop. TREEHILL is prepared the same way and is a filter for the same reason: with the crop box at $x \in [-2.3, 46.2]$, $y \in [-16.3, 7.8]$, $z \in [-21.5, 28.4]$—the axis-aligned box around what is in front of the camera and inside the renderer's far plane—2 074 009 of the released 3 783 761 Gaussians clear the floor inside it, under the 2.1M budget, so once again the seeded draw never binds and the file is rendered at degree 3. Its floor, however, is a parameter of the measurement rather than a cleanup step, and we set it apart because of that: it is 0.005, and it is load-bearing. The semi-transparent veil *is* the low-opacity Gaussians, so raising the floor deletes the phenomenon the scene was chosen for—the partial fraction over the flame's silhouette falls 93.8 → 84.2 → 79.6 → 65.2 → 55.0% at floors 0.005, 0.02, 0.05, 0.10 and 0.20, so KITCHEN's 0.35 would leave nothing here to measure. What the crop does not do is open a hole: mean rendered alpha at the shipping camera is 0.93780 for the raw release and 0.93780 for the prepared asset, identical to five decimal places, and the prepared asset reproduces the raw one at 89.9 dB. The campsite is reduced harder: 380k Gaussians at degree 1, a truncation we report as a limitation rather than a choice, since it flattens exactly the view-dependent shading a reconstruction is

trained to produce. The opacity floor is the only lossy step not forced by the renderer, and on KITCHEN it is the only lossy step at all besides the crop: a trained reconstruction carries a haze of near-transparent floaters which, once a subsample thins the surface behind them, read as speckle rather than atmosphere. Every scene figure in this paper states the threshold and degree it was made with, and the tool that performs the step is in the supplemental, so the preparation is reproducible even where the input is not.

One consequence of a real capture is worth stating separately, because the rest of this paper is dimensionless. A reconstruction has no intrinsic scale, so we set it from an object in the scene. On the campsite the vehicle does it: a body length of 4.9 m fixes 5.31 source units per metre, which in turn fixes the flame at roughly 1.2 m, and it is that calibration the combustion figures of §5.7 are quoted in. Scene figures here are therefore sized against a real object rather than eyeballed—the first place in this work where a world unit means anything.

5.6 Scene Composition and Secondary Effects

The environment is itself Gaussian content: either a procedurally scattered dark scene (640 flattened ground and enclosure splats) or a reconstructed 3DGS capture loaded from a standard Inria `.ply` (scales exponentiated, opacities sigmoided, and colour from the DC term plus spherical harmonics up to degree 3—bands 0–1 in vertex attributes, bands 2–3 fetched from a per-splat texture, evaluated per frame against the view direction; subsampled to a splat budget when the prepared file exceeds it—800k here, which KITCHEN's 729 826 splats do not reach (§5.5), against 380k for the campsite, which does; the interactive demo defaults to 120k). Scene splats are rasterized with the standard EWA forward projection [Zwicker et al. 2001]: the full 3×3 covariance is carried per instance, projected to a screen-space conic through the view rotation and perspective Jacobian in the vertex shader, and the conic's eigenvectors span the quad. Procedural environment splats join the sorted buffer of §5.3 directly, giving per-primitive mutual occlusion with the fire. Loaded reconstructions are merged into the same global depth order by *depth bucketing*: each frame, scene splats are counting-sorted into K=1536 view-depth buckets in $O(n{+}K)$, and the renderer walks the buckets near-to-far, interleaving ranges of the (exactly sorted) dynamic stream with ranges of the bucketed scene stream under the shared blend state, coalescing adjacent same-stream ranges to minimize state changes. Order is thus exact within the dynamic stream and bucket-granular between and within the streams. Within the dynamic stream that ordering is strictly finer than the per-slice compositing granularity of layered production fire [Horvath and Geiger 2009]; between the streams it is a claim about the loaded scene rather than about the method, because the K buckets span whatever view-depth range the reconstruction occupies. They are uniform in *linear* view-space depth, so they are of equal width everywhere rather than widest near the camera, and the dynamic stream is bucketed by the same key—a point worth stating because an earlier build keyed the two streams in different units (squared eye distance for the fire, depth for the scene), which pushed every fire particle into the farthest bucket and drew it behind the whole scene. The granularity is bounded rather than asserted. We built an order-exact reference—a single merge of the two streams by primitive depth, with no bucketing anywhere—inside this same renderer, and measured the bucketed path against it over four cameras and K from 24 to 65 536, 60 instrumented runs in all. Two things hold across every one of them. The displacement is bounded by the bucket: the largest depth by which any primitive is drawn out of true order is 0.999961 of one bucket width and never as much as one, so no primitive can cross another that is more than a bucket away. And the residual falls as $1/K$—the count of cross-stream inverted primitive pairs scales with fitted exponents -0.989 to -1.011 over that range. In image terms, at the shipping K=1536 the interleave alone (scene stream exactly sorted, only the merge bucketed) reproduces the order-exact reference to 63.3–79.3 dB PSNR, mean absolute error 0.0007–0.008 of one 8-bit level, with essentially all of the residual on soft edges rather than spread over the frame; with the scene's own $O(n{+}K)$ counting sort in place as well, which is what actually ships, it is 49.2–65.1 dB. The bucketing is therefore a cost-accuracy knob with a known bound, not an unquantified approximation. On a capture the relevant question is whether a bucket near the fire is coarser than the stack's per-slice spacing D/N, and on KITCHEN we can now answer it: the reconstruction spans 39.5 view-depth units, so a bucket is 0.026 units wide, and the flame occupies buckets 616–677—62 buckets for 9 slices, about 7× finer than the slice spacing. The bucketing's overhead is small at small scene sizes and not negligible at large ones: with the interleave on vs. a draw-after pass we measure 22.9 vs. 21.2 ms median render time on the synthetic test scene (45.8k splats under the 9×1300 flame; +1.7 ms, +8%) and 291.0 vs. 239.3 ms on KITCHEN (729 826 splats under the shipping 9×600 preset; +51.7 ms, +22%). Per scene splat that is 38 against 71 ns, so the premium grows with scene size rather than staying flat, and it is dominated by the per-splat scatter into bucket order that the draw-after path skips entirely—not by K, which we swept separately (K=96/384/1536/4096 gives 22.7/22.7/22.9/23.4 ms at 45.8k, and K=96 vs. 1536 gives 284.4 vs. 291.0 ms at 729 826) (Figure 10; measurement protocol in §7).

In-scene ignition. Because the fire shares the scene's depth order rather than sitting in front of a backdrop, a source can be lit *on a reconstructed surface*: the plume then rises inside the captured geometry and is occluded per primitive by the objects around it (Figure 10, top left). We state the compositing claim precisely, and more narrowly than we did before measuring it. A backdrop system *can* recover much of this occlusion by testing the volume against a per-pixel depth buffer of the scene; we implement that stronger baseline (Figure 10, top right), and where the occluder is opaque it is right—on KITCHEN it matches our composition to within 0.4% of the fire's contribution, and we report that rather than claim a win we did not measure. What per-pixel depth cannot express is mutual occlusion among *semi-transparent, interpenetrating* primitives—fire splats threading between a foreground post's splats and translucent smoke wrapping partially charred geometry—where no single depth per pixel exists, so the z-test degenerates to a binary cut at every soft boundary. Per-primitive interleaving resolves those cases by construction, and in-place authoring falls out of it. Earlier versions of this paper had to leave that last claim as an argument, for want of a captured scene with anything translucent in front of the fire; it is now measured, on a second public benchmark reconstruction

whose foreground foliage is genuinely semi-transparent (Figure 11, §5.5). Over that flame's silhouette 88.1% of pixels receive partial transmittance under our composition and 9.1% under the strongest depth-aware baseline at the threshold that matches our total fire energy; the baseline reaches its mean by splitting the same silhouette into 51.5% fully open and 39.4% fully blocked, which is the predicted binary cut and not a tuning failure. The two captures together are the result: the advantage appears exactly when the occluder is semi-transparent, and vanishes when it is binary. Ignition composes with the fire-spread loop of §5.7, which turns a single lit point into scene-wide, multi-source fire.

The fire acts as a light source on the scene through an inexpensive screen-space model: environment splats are warmed by a radial falloff around the flame's screen position with a floor-proximity boost—a deliberately cheap stand-in for volumetric light transport that nevertheless anchors the flame in the scene (§8). Sparse ember particles (velocity-aligned, twinkling Gaussians in the shared sorted pass), plus two screen-space effects—a mirrored planar ground reflection and an FBM heat-haze refraction above the flame—complete the presentation; all are optional.

5.7 Fire Spread on the Scene's Own Primitives

Because the environment is itself a set of point primitives, combustion state can live *directly on the reconstruction*: each scene splat carries heat h, fuel f, a burning/charred state, and a glow amount. Fuel is the one piece of that state a capture cannot supply—a reconstruction carries no material labels—so it is authored per scene by a geometric rule (§5.7). Adjacency is a spatial hash over the prepared splats as loaded (§5.5), built once in the reconstruction's source frame (invariant to scene placement) with a cell sized from the spread radius rather than from the scene's extent, and per-frame work is restricted to the fire front—burning splats and their neighbors—capped at a fixed budget, plus an $O(n)$ heat-decay scan.

The dynamics close a loop between the simulation and the scene. *Scene ignition:* sampled hot flame particles deposit heat into nearby splats (h decays, so ignition requires sustained heating); past a threshold a flammable splat ignites. *Spread:* burning splats heat unburnt neighbors with a buoyant upward bias, following the standard heat-driven spread models of interactive wood combustion and mesoscale wildfire [Hädrich et al. 2021; Pirk et al. 2017]—the spread physics is deliberately conventional; per-splat fuel supports non-flammable regions. *Consumption:* fuel depletes over a burn time τ; while burning, an HDR ember-glow term is added in the scene shader, and a char fraction drives the splat's albedo toward a desaturated soot value at unchanged opacity—combustion is written directly into the rendered attributes of the captured primitives. *Scene-to-simulation:* burning splats spawn hot seed particles in the nearest slice (round-robin, budgeted), which enter the temperature mixture (Eq. 3) directly, so ignited geometry grows its *own* plumes through the full baroclinic loop. The per-frame budgets that bound the loop's cost are fixed: at most 160 flame-particle heat samples, 48 scene-to-simulation seed emissions, and 1400 frontier splats processed per frame. On the synthetic test scene that is enough to make the cost flat in how much of the scene burns. It is *not* enough on a capture, and we report the failure rather than the design intent: the cap bounds how many burning splats are processed, but not what each one costs, because the neighbourhood query it performs scales with local splat density—and a reconstruction is far denser, and far less uniformly dense, than a scene we scatter ourselves (§5.7).



Fig. 4. The combustion loop on a captured reconstruction (§5.7). Fuel is assigned geometrically—splats between 0.25 and 2.45 world units above the ground and inside the vehicle's footprint—a $5.0 \times 1.7 \times 2.1$ m box, 44 861 of 380 000; the dirt is not flammable. A single ignition of 40 splats at the flank spreads through the body: *left* (t=35 steps) 2 591 splats burning and 7 000 charred; *middle* (t=85) 2 068 and 18 392; *right* (t=165) 1 611 and 37 800. Char is written into each splat's own albedo, so the burnt region is darkened reconstruction rather than an overlay, and the plume is fed by seed particles the burning splats inject into the solver—the same closed loop as Figure 6, on geometry nobody authored for it. The cost, however, does not carry over: at this splat density a 1/60 s substep costs 99 ms in the left panel's state—six times slower than real time—and §5.7 reports why the frontier cap does not bound it.

Figure 6 shows the loop end to end on the synthetic test scene: a single ignition inside the fire ring spreads through the fuel, the burning splats glow in HDR and darken as they char, and—because they inject temperature and particles into the nearest slice—the plume above the fire is fed by the *scene*, not only by the original emitter. With a ~190-splat active fire line the frame stays comfortably real-time (render 4.6 vs. 3.9 ms with the fire line active vs. quiescent, same machine as Table 4). Whether ignition can cross a fuel gap wider than the adjacency radius we now test directly rather than speculate about: in a two-island configuration on the synthetic scene (fuel zeroed in an x-interval; floor and wall non-flammable), ignition never crossed gaps of 0.12–0.65 world units (1.1–6× the adjacency radius) in 35 simulated seconds, at winds up to 2× the default—the burning island chars completely (5 465 splats) while the far island stays untouched. The mechanism is buoyancy: hot particles rise at ~1.1 units/s against a lateral wind of 0.35–0.6, so by the time the plume has drifted across even the narrowest gap it is above the far fuel's height and outside the flame-to-scene heating radius. These distances and speeds are in that scene's own units and are not metric: a loaded scene is auto-scaled on import to a common world span on import, so a world unit is a fixed fraction of a scene's extent rather than a length, and only the capture of §5.5 is calibrated against a real object. What transfers is the ratio to the adjacency radius, not the absolute gap. The crossing we observed in an earlier, more compact configuration reached a *tall* object standing inside the slice slab—consistent with the same mechanism, not an exception to it—so gap crossing in this system is a statement about target geometry, and lateral ember-mediated ignition (embers are currently visual-only) is the natural mechanism to add for it.

The loop on a captured reconstruction. The same loop runs on the campsite capture of §5.5, which is the first time any primitive



Fig. 5. The same loop on the *public benchmark* scene (§5.7)—the demonstration a reader can actually repeat, since KITCHEN can be downloaded and the campsite cannot, and the one place in this paper where *nothing is ignited by hand*. Fuel is two hand-drawn boxes: the model construction loader on the table (75 443 of 729 826 splats) and the tabletop strip its blade rests on (25 430), which is what the flame actually stands in. *Top left* (t=11.95 s): the strip is alight (2 788 splats burning, 1 400 already charred) and the machine has received *exactly zero* heat—it is 0.59 world units from the flame against a 0.14 heating radius, so the flame cannot touch it and the front has to come across the table. *Top right* (t=24.40 s): it catches: the first 29 machine splats cross the ignition threshold. Heat first registers on the machine at t=13.94 s ($h_{\max}$=0.027), so the induction is 10.5 s of measurable heating, 24.40 s from the flame being lit—and nothing schedules it, because the system has no delay parameter. Those 29 splats of 729 826 are the measurement and not yet the picture; the char takes another minute to become unambiguous, for the per-pixel reason the shading paragraph quantifies. *Bottom left* (t=99.83 s): 3 154 burning and 6 235 charred, 12.45% of the machine—a fire line climbing the blade and lift arms while the cab and tracks are still yellow, the strip by now spent. *Bottom right* (t=249.73 s): 41 192 charred and 7 565 still burning, 64.63% of the machine; the run is halted here, not burnt out. Char is written into each splat's own albedo, so this is darkened reconstruction, not an overlay, and it now reads as soot rather than the dark olive of earlier versions: over the 57 252 pixels that char, saturation falls 0.909 → 0.202 and mean colour sRGB$(207, 187, 19)$ → $(72, 66, 57)$, while the blade ribbing and stud relief survive (§5.7). Own it: the strip burns too, and it is meant to—this is a fabric-to-plastic hand-off, not a flame licking a target. The blade teeth stay yellow because they lie below the fuel box's floor, which the tabletop plane forces. One continuous run of 15 044 substeps, ~45 minutes of wall clock; containment held at all 56 checkpoints, zero burning or charred splats in five probe boxes outside both fuel volumes.

from a real reconstruction has burned in this system, and it is worth separating what transferred from what did not.

What transferred is the behaviour. Fuel is a per-splat quantity, and a capture carries no semantics, so we assign it geometrically: every splat inside a $5.0 \times 1.7 \times 2.1$ m box around the vehicle, starting 0.25 world units above the ground plane—44 861 of 380 000—is flammable, and the dirt is not. Deciding what burns is the part a capture does not help with: the box is drawn by hand, and doing it from the reconstruction alone is a semantic-segmentation problem this work does not address. A single ignition of 40 splats at the vehicle's flank then spreads through the body under the same rules as the synthetic scene. Char is written into the splats' own albedo, so the burnt vehicle is dark reconstruction, not a decal over it, and the plume above it is fed by seeds the burning splats inject into the solver (Figure 4).

What did not transfer is the cost bound. We time the loop by difference on the same loaded scene, and quote everything per simulation substep ($\Delta t = 1/60$ s), the unit the loop actually advances in; both captures were measured back to back in one session. Disabling the loop and leaving everything else alone costs 31 ms per substep on the campsite and 32 ms on KITCHEN's 729 826 splats; enabling it with nothing yet burning costs 33 ms on both, so the $O(n)$ heat-decay scan is ~2 ms and is not what hurts. The fire line is. Replaying both recipes to the state Figure 4's left panel is shot in, ~2.5k splats burning, a substep costs 99 ms on the campsite—the frontier alone is 66 ms, 27 μs per burning splat, and the loop runs six times slower than real time—and 146 ms on KITCHEN at the same instant with ~2.8k burning, 41 μs per burning splat: half again as expensive per splat processed for 1.9× the scene. The frontier cap is doing its job—it bounds the number of burning splats processed—but each of those performs a neighbourhood query whose cost is the local splat density, and on the campsite a query radius that returns 81 true neighbours scans 405 candidates through the adjacency hash. (It scanned 51 035 before we sized the hash cell from the spread radius rather than from the scene's extent, a mis-sizing that was invisible on a scene whose extent happened to be a few world units. The hash is an accelerator only—queries verify $d^2 \le r^2$ regardless—so this changed no result, only the constant.) The honest summary is that the compositing half of our scene story is real-time on a real capture and the combustion half is not: at these densities the loop is an offline effect, and making it interactive on captured geometry needs either a coarser burn representation than one state per splat, or the same frontier work moved to the GPU.

The same loop on a scene the reader can download. The campsite is a capture we can neither redistribute nor re-shoot, which makes it the wrong place to leave this result: nobody outside can check it. We therefore ran the identical loop on KITCHEN (§5.5), in the same placement the teaser and the occlusion comparison use, with nothing moved to make the burn work (Figure 5). Fuel is again a hand-drawn box, here in the demo's world frame around the model construction loader standing on the table—$x \in [0.28, 2.90]$, $y \in [0.095, 2.20]$, $z \in [-2.15, 1.70]$, 75 443 of 729 826 splats—together with a second, thin box over the tabletop strip that the flame stands in and the blade rests against ($x \in [-1.60, 3.20]$, $y \in [0.00, 0.12]$, $z \in [-1.30, 0.45]$, 25 430 splats, 260 of them shared with the first). The machine box is character-for-character the one this figure used when it seeded ignition by hand; the strip is the change, and it is the whole

result. Purity matters more here than on the campsite, where the vehicle stood alone on dirt: this machine rests on a mat on a cluttered table, so we checked the selection three ways—the subset alone renders as the machine and nothing else, its complement renders the scene with the machine gone, and 26-connected labelling puts 73 346 of the 75 443 in one component—and asserted containment during the run itself, which reports zero charred splats outside the box at every checkpoint. What the box cannot reach is the ∼2% of the machine that interleaves with the tabletop: the tabletop is a hard plane at $y = 0.070 \pm 0.012$ and the blade teeth and track undersides run down into it, so a floor low enough to catch them sweeps in 47 002 table splats to gain ∼1k machine ones. We kept the clean box, and the blade teeth stay yellow in the figure. With nothing lit by hand anywhere in the run, the front crosses to the machine at $t = 24.40$ s and then consumes it steadily: 170 splats lit at $t = 40$ s, 1 039 at 60, 9 389 (12.45%) at 100, 21 934 (29.07%) at 150, 35 893 (47.58%) at 200, and 48 757 at $t = 249.73$ s—64.63% of the machine, 41 192 of them already charred—which is where we stop and shoot the last panel. We should be plain about what that endpoint is: the fire is still burning there (7 565 splats on the frontier, 34 251 of the machine's fuel unconsumed), so this run is halted, not burnt out, and we report the state we measured rather than extrapolating to one. Reaching it took 15 044 substeps and ∼45 minutes of wall clock, a mean of 179 ms per 1/60 s substep—eleven times slower than real time. That is the honest headline of this second demonstration: the loop closes on a scene anyone can download, and it is an offline effect there too.

The charring rate is set by the frontier cap rather than by the scene, and this run shows it directly. At most 1400 burning splats are burnt down per substep, so the *absolute* rate saturates: charred splats accumulate at 210–245 per simulated second over essentially the whole run, flat against a fire line that grows by a factor of four underneath it. What differs between the two captures is therefore the fraction of the target consumed and the wall clock, not the speed of the front. The frontier itself stood at 1 893–8 041 splats—1.4 to 5.7× the cap—so the cap was binding throughout while the frontier list grew unbounded beneath it, which is exactly the failure mode the previous paragraph measures.

The machine lights itself, and how long that takes. This figure used to seed ignition with an explicit op, and that was a shortcut hiding a geometry error. As shot, the emitter's base stands 0.59 world units from the nearest fuelled machine splat against a flame-to-scene heating radius of 0.14—four times too far—so no flame particle was ever within reach of the machine, and the fire could not have lit it however long it ran. The remedy is geometry and not a timer: we fuel the tabletop strip the flame already stands in and the blade already rests on, which makes the material continuous from the flame's own footprint to the machine, and then we ignite nothing at all. The flame lights the strip inside the first second (nearest fuelled splat 0.023 world units from the base, comfortably inside the heating radius). The machine's splats then read *exactly zero* accumulated heat through $t = 11.95$ s— confirming directly that nothing the flame itself does reaches them—after which heat begins arriving from the burning strip and competes with the loop's $0.35\,\mathrm{s}^{-1}$ leak: the next sample, at $t = 13.94$ s, is the first with any heat on the machine at all ($h_{\max} = 0.027$ over the 200 splats nearest the flame), and the first 29 machine splats cross $h \geq 1$ at $t = 24.40$ s. The induction is thus 10.5 s of measurable heating and 24.40 s from the flame being lit, and it is emergent: the system contains no delay parameter and the recipe contains no ignition op. The ramp is visibly sawtoothed—$h_{\max}$ runs $0.564 \to 0.550$ and $0.701 \to 0.667$ across consecutive half seconds on its way up— because the leak competes with a front that only advances when the 1400-splat budget frees up. That is the frontier cap of the previous paragraph showing up as a *shape* rather than as a rate. Re-running the same configuration during the design of the fuel selection reproduced this trace exactly—the same $h_{\max}$ at every sample through the ignition window and the same 29 splats at the crossing—so the induction time is a property of the configuration and not of a particular run.

This does not contradict the two-island result above, and the difference is worth one sentence. That experiment deleted the fuel between two islands and found the flame could not bridge a gap it had to cross through the air; this one deletes the gap instead and leaves a continuous fuel path with a material boundary in it. What crosses here is the splat graph, not the flame—which is precisely the claim the loop is supposed to support, now demonstrated without a hand-placed ignition anywhere in the run.

What char should look like, and why the obvious operator does not. Charring writes into the splat's own shading, and the operator we shipped for it was wrong in a way worth reporting, because it is the obvious operator and the closest prior system uses it as well. We had scaled the charred splat's albedo toward black in proportion to the char fraction and dropped its opacity by up to 30%; FieryGS likewise dims the colour of each Gaussian by a factor driven by relative char mass [Shen et al. 2026]. Decomposing those two halves against one identical simulation state on this scene, over the 11 522 most deeply charred pixels, the opacity drop moves the composited colour by less than 1/255 and *raises* luminance: the entire defect is the multiply. A multiply preserves linear chromaticity exactly, so it slides a colour down its own ray—and where the tone curve had been rolling this scene's saturated yellow off toward white, dimming pushes it back down into the linear part of the curve where the full chroma ratio survives. Measured saturation therefore *rises* under charring, $0.513 \to 0.640$, and a dark saturated yellow is olive. Soot is not a dimmer; it is a deposit that replaces the albedo. We desaturate to luminance and then dim, $\mathrm{mix}(c,\ \mathrm{grey}(c),\ 0.85) \cdot 0.12$, and leave opacity alone, because soot lies on a surface and does not make it transparent—which takes the same pixels to 0.141 saturation and reads as soot. The check that this darkens rather than deletes is relief: luminance gradient over the charred region holds at 0.78× the uncharred render, against 0.85× for the old multiply, where mixing straight to a flat soot albedo—which lands closest to a soot colour and is the tempting choice—collapses it to 0.39× and punches a visible hole through the machine at figure scale. The change is in the scene fragment shader only; all three determinism anchors are bit-identical across it.

We scope the claim precisely: heat-threshold spread with fuel and charring is long established on meshes and voxel grids [Hädrich et al. 2021; Pirk et al. 2017], and FieryGS spreads fire between

contacting objects by solid heat diffusion on its occupancy grid, offline [Shen et al. 2026]. What is new here is the *substrate and the loop*: combustion state carried on the splats of a captured scene, where the forward-simulated flame both ignites the reconstruction and is re-seeded by it—in real time on the synthetic scene, and, on the capture, at the seconds-per-step cost measured above. One honest caveat: the loop projects between world space and slice coordinates through the current slice frame, so under view-aligned slicing with a moving camera the spread coupling inherits the view-adaptation of §4.3; the world-fixed mode (or freezing the frame during spread) removes this dependence (§8).

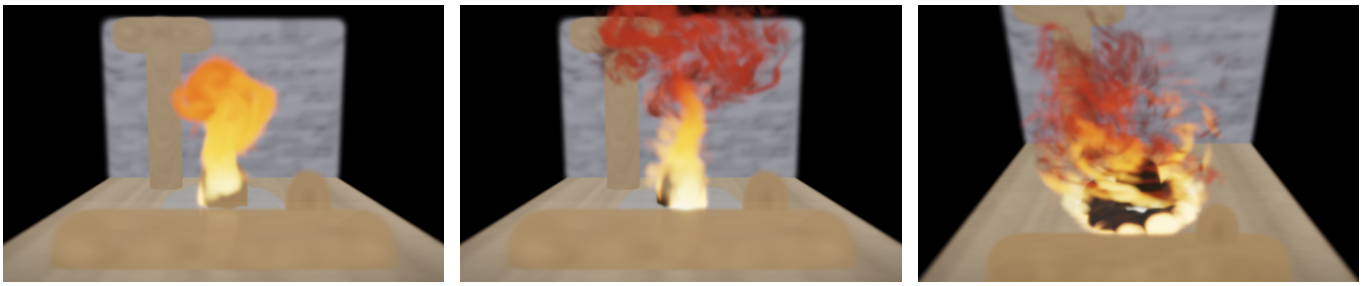


Fig. 6. Fire spread on the scene's own primitives (§5.7), one continuous simulation on the *synthetic* campfire test scene—the redistributable one; the same loop on a captured reconstruction is Figure 4. A single ignition (4083 splats, radius 0.35) is seeded inside the stone ring at t=0; fuel is assigned to every splat below the wall top (41 020 of 45 808), and the stone ring and wall carry none. *Left* ($t\approx$2 s): the ignition is still local—rear post, fuel logs and ring are intact. *Middle* ($t\approx$16 s): the front is still inside the ring—the first char has blackened the near side of the flame's base, while the post beside it and the nearest log are untouched; burning splats glow in HDR and inject temperature and particles into the slice solvers, so the plume above is fed by the scene rather than by the emitter. *Right* ($t\approx$56 s): 12 600 splats charred and 1762 still burning, with char written directly into splat albedo—the black remains filling the ring, with the unfuelled ring stones still pale around them, are scene geometry rather than smoke. Shown from a raised camera so the char is not hidden behind the flame's bright core.

6 Implementation

The entire system—simulation, sorting, rendering, UI—is a single dependency-free HTML file: simulation and per-frame depth sort run in JavaScript on the CPU; rendering is WebGL2 with instanced quads, an RGBA16F HDR target (8-bit fallback), and hand-written camera matrices. We emphasize what this implies: the method needs no GPU compute, no CUDA, and no precomputation, bake or training step on the simulation side, and still reaches real-time rates (§7). The scene side is the exception, and it is offline rather than absent: a trained reconstruction is prepared once before it can be loaded (§5.5), a data-reduction step that touches no solver state. Two named configurations are used throughout and every reported number is tagged with one of them. The **Interactive preset**—the shipping default—is N=9 layers × 600 particles at θ=0.9 (~5.4k flame Gaussians, ~60 FPS end to end). The **Quality preset**, used for all accuracy-critical measurements in §7 unless stated otherwise, is N=9 × 1300 at θ=0.7 (~11.7k flame Gaussians), 640 procedural environment splats, ≤160 embers—about 12.3k sorted Gaussians per frame, plus the loaded scene when there is one: 45.8k splats for the synthetic test scene, 729 826 for KITCHEN and 380k for the campsite (§5.5), merged into the global depth order by $O(n)$ bucketing (§5.6). Table 3 lists the main parameters. A headless harness drives the demo without a display and reads back probe images for the quantitative measurements in §7. For reproducibility, every stochastic simulation path (emission jitter, ember spawning, initial conditions) draws from a single seeded PRNG stream; combined with a fixed Δt, a run is fully determined by `(seed, steps)`, and an FNV-1a hash over the complete simulation state verifies bit-exact replay (§7).

Table 3. Key parameters (defaults). World domain 2 × 3 units. These are solver units and carry no physical scale on their own: only a scene calibrated against a real object supplies one, and the single calibrated scene in this paper (5.31 source units per metre, §5.5) puts the flame at roughly 1.2 m. Every other length and speed we quote in world units is dimensionless.

symbol	meaning	default	section
N	slice layers	9	§4.3
N_p	particles per layer	1300	§4.1
σ_0	vortex core radius	0.05	Eq. 6
θ	treecode opening angle	0.7	§4.2
ν	core-spreading viscosity	0 (opt.)	§4.1
N_{dir}	director super-Gaussians	≤16+16	§4.2
β	baroclinic gain	1.4	Eq. 4
β_b	direct buoyancy	0.9	§4.1
κ	cooling rate	1.2	§4.1
c	global (coarse) coupling	0.5	Eq. 7
κ_z	inter-slice coupling	0 (opt.)	Eq. 8
D	stack depth	0.7	§4.3
k_Σ	covariance gain	1.3	Eq. 9
$\kappa_{\max}$	eigenvalue clamp (bounds axis ratio)	3.6	Eq. 9
—	emissive gain / bloom / threshold	1.4 / 1.0 / 0.95	§5.4

7 Results and Evaluation

Deterministic replay. All measurements below are taken through a deterministic-replay harness: every stochastic simulation path draws from one seeded PRNG stream, the solver is stepped at a fixed $\Delta t = 1/60$ s, and an FNV-1a hash over the complete particle state (position, circulation, core radius, both temperatures, density, velocity), embers, and—when the retained grid baseline runs—its coarse field, fingerprints each frame. The treecode is hashed only through its effect on particles: it is derived data, rebuilt from the particles every frame. Two runs from the same seed produce bit-identical state hashes after 120 steps (this holds for both the grid-free solver, whose Morton-ordered build and fixed traversal order are deterministic by construction, and the baseline); a different seed

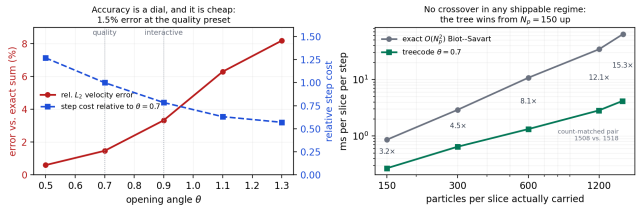


Fig. 7. The treecode, priced against the exact sum it replaces. *Left:* relative L_2 velocity error vs. opening angle θ (error is deterministic and environment-independent), with step cost relative to θ=0.7 measured in one browser session (the absolute anchor is Table 4); the two presets are marked. *Right:* per-slice cost of the treecode vs. the exact $O(N_p^2)$ Biot–Savart sum (3-seed medians, same session as Table 4), plotted against the count each run actually carried rather than the budget requested. The four lower points are count-matched to within 0.3%. At the largest budget requested (2400 per slice) neither path sustains the request under the pool cap, and their medians land 21% apart in count, so we plot instead the one seed pairing whose two runs are comparable—1508 particles for the tree against 1518 for the exact sum, within 0.7%, and the tree still 15.3× faster. Correcting the mismatched median pair for count instead, by N^2 or by the measured $N^{1.79}$, gives 13–15× over the same range. The tree is faster at every count we can ship, from 3.2× at N_p=150 to 15.3× at ∼1.5k—the quadratic wall is removed without a grid.

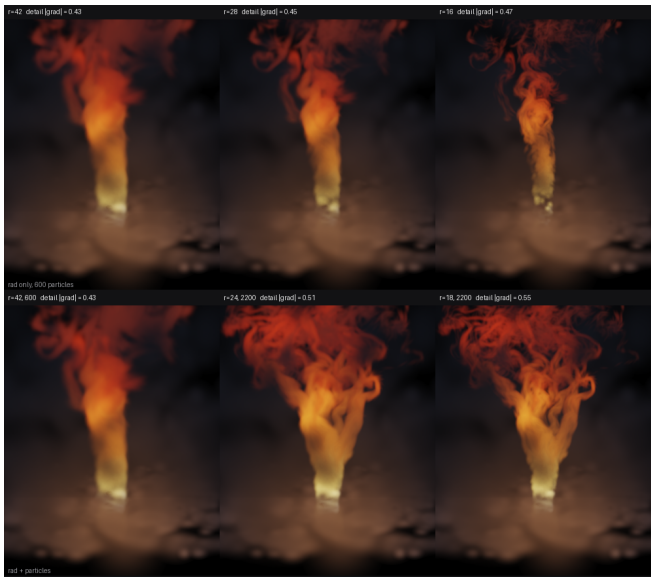


Fig. 8. Perceived detail is bounded by the particle budget, not by a rendering knob. Same seed, same step count, same shading; the number is mean image gradient. *Top:* shrinking the on-screen splat radius at a fixed 600 particles per layer is free—the radius never enters the solver—and does sharpen the image (0.43 → 0.47), but past ∼22 the blobs stop tiling and the plume breaks into visible granules: the overlap condition $h \lesssim \sigma_0$ failing in the picture rather than in the theory. *Bottom:* spending particles instead lets a smaller radius still tile, and reaches detail the radius alone cannot (0.51 at $2200/r$=24, 0.55 at r=18).

diverges—so every number in this section is exactly reproducible from a seed and a step count.

Performance. Table 4 reports per-frame timings, measured over 90–240 simulated and 60 rendered frames per configuration at 1280×800 on a workstation (AMD Ryzen Threadripper PRO 3945WX; NVIDIA GeForce RTX 3080 via Chrome/ANGLE D3D11; simulation single-threaded in CPU JavaScript), median of five seeds per configuration (three for the exact-sum crossover sweep of Figure 7), all captured in one scripted browser session. Grid freedom is not free, and we price it precisely. Against the retained vortex-in-cell baseline at matched counts (N=9, N_p=1300, θ=0.7), the grid-free solver's simulation step costs 2.3× more (29.8 vs. 12.9 ms)—the cost of evaluating ∼10^6 closed-form pair and cluster interactions per frame instead of one 32×48 relaxation per slice—while remaining interactive end to end. At the interactive preset (N_p=600, θ=0.9) the premium shrinks to 1.3× (10.4 vs. 7.8 ms) and the full pipeline runs at ∼60 FPS. What the extra milliseconds buy is measured in the spectral A/B below: velocity detail no longer capped at a grid Nyquist, and a solver whose state is exactly the picture. The same harness times the exact $O(N_p^2)$ Biot–Savart evaluation the treecode approximates (Figure 7). There is no crossover in any regime we can ship: the treecode is already 3.2× faster at 150 particles per slice, and the gap widens to 8.1× at 600, 12.1× at 1200, and 15.3× at ∼1.5k (4.2 vs. 63.6 ms per slice) as the exact sum grows quadratically, while the treecode grows as $N_p \log N_p$ with a measured relative L_2 velocity error of 1.5% at θ=0.7 (0.6% at θ=0.5, 3.3% at θ=0.9, 6.3% at θ=1.1)—the quadratic wall is removed without a grid, and without an unmeasured approximation.

We choose the N_p=1300 default deliberately: the criterion is the standard vortex-method overlap condition—mean inter-particle spacing h below the core radius σ_0, so the blobs tile the plume into a continuous field—plus image quality (a sparser population no longer reads as a flame). Image quality, not frame time, sets this budget.

That the two criteria coincide is worth showing rather than asserting, because it bounds what rendering alone can buy. The on-screen splat radius is a pure rendering parameter—it enters the instance packing and nothing in the solver—so shrinking it is free, and it does sharpen the image: at a fixed 600 particles per layer, dropping the radius from 42 to 16 raises mean image gradient from 0.43 to 0.47 and thin structures start to appear. But it cannot be pushed, because the blobs stop tiling: below ∼22 the plume visibly breaks into discrete granules, which is the overlap condition failing in the image rather than in the theory. Buying the same sharpening the other way—more particles, so a smaller radius still tiles—reaches 0.51 at N_p=2200, r=24 and 0.55 at r=18, beyond anything the radius alone attains (Figure 8). Perceived flame detail is therefore bounded by the simulated element count, not by a free rendering choice: it is a scaling property of the representation, and the honest way to buy more of it is a faster solver—a GPU port of the same closed-form kernels—rather than a shading trick. For calibration of the design point (not of solver efficiency—these systems integrate far larger simulation state), the closest prior systems report 2.37 s per frame (FieryGS [Shen et al. 2026], RTX 4090D) and 38–228 s per *time step* (Gaussian Fluids [Xing et al. 2025], RTX 4090); production layered fire required ∼20 s per slice per frame at ∼2048^2 slice resolution on

a GPU farm [Horvath and Geiger 2009], versus our ∼1300 particles of core σ_0=0.05 per slice (the retained grid baseline runs 32×48)—much of the gap is resolution deliberately traded away, the rest is architecture.

Why we do not normalize across systems. It is tempting to divide those wall-clock numbers by a hardware factor and declare a winner. We refuse to, and the arithmetic says why. Between our RTX 3080 and the RTX 4090 those systems used, peak FP32 rises 2.77× while memory bandwidth rises only 1.33×; no scalar correction exists without an arithmetic intensity none of the three papers publishes, and our own simulation runs single-threaded in interpreted JavaScript on the CPU, so a GPU ratio has nothing to multiply. Normalizing per degree of freedom inverts the conclusion rather than settling it: against FieryGS's 256^3 grid our cost per element per frame is roughly an order of magnitude *worse*, while by wall clock we are roughly two orders *better*. Both statements are true and neither is informative, because a grid cell and a Lamb–Oseen blob are not the same unit of work, of state, or of resolvable detail. Benchmarking practice in high-performance computing reaches the same conclusion—speedup is not a valid instrument for comparing different algorithms on different machines [Hoefler and Belli 2015]—so we report absolutes with their full context throughout and never average a ratio.

What we normalize instead is *within* this system, where the units are identical and the number is therefore meaningful. Simulation cost per particle per frame is flat at 2.1–2.7 μs across a 17× span of element count, which turns the slice sweep of Table 4 from a list into a scaling claim, and incidentally exposes where the grid baseline's advantage comes from: its cost per particle falls from 1993 to 995 ns as N_p rises, because the fixed 32×48 relaxation is being amortized over more particles—the grid is cheap in the limit where it is coarse relative to the detail you asked for. Rasterization is likewise predictable, and closer to a single rate than we first thought: the slice sweep gives 570 ns per Gaussian per frame, while the paired scene measurements below put the *marginal* cost of a scene Gaussian at 365 ns at 45.8k, 375 ns at 3.8×10^5 and 373 ns at 7.3×10^5. The ∼0.37 μs figure is a working constant—flat to within 3% over one and a half orders of magnitude in scene size—which is what makes the cost of loading a reconstruction a quantity a reader can bound rather than a surprise, provided it is read as a rate for scene splats specifically and not as one rate for every Gaussian on screen. The one machine-independent rate we can state for the solver is throughput: ∼33.6 M treecode interactions per second, single-threaded.

Comparison to external systems. We run no external system head-to-head, and we say precisely why rather than leaving it as an absence. Of the systems that could be compared, only FieryGS [Shen et al. 2026] addresses the same problem—forward fire inside a Gaussian-splatting scene—and its published configuration is $\sim10^3\times$ our simulation state on hardware we do not have, with neither render resolution nor scene Gaussian count reported, so a wall-clock comparison would produce a number without a meaning. The remaining physics-for-GS systems (PhysGaussian, Gaussian Splashing, VR-GS [Feng et al. 2025; Jiang et al. 2024; Xie et al. 2024]) simulate solids or liquids and model no gaseous or emissive medium at all, so there is nothing in them to compare our fire against; that the real-time column and the gaseous column are disjoint in the prior literature is itself the clearest statement of the gap this work fills.

What we *can* compare, and do, is architecture. FieryGS composites its fire over the reconstruction as an opaque background—the phrase is its own, and its rendering equation adds fire and smoke radiance in front of a transmittance-weighted Gaussian-splatting term—which is exactly the backdrop baseline of Figure 10, implemented inside our own renderer with our own shading, on a reconstruction we neither captured nor trained (§5.5), so that the only variable is the compositing structure. The occlusion figure is therefore not a strawman we invented but a faithful reproduction of the state of the art's composition model (the renderer is shared, the scene is not ours, and the stronger depth-aware variant beside it is swept rather than tuned to lose—indeed it is swept and it *ties* with us on that opaque occluder, and on the semi-transparent one of Figure 11 we go further and hand it the threshold that matches our fire energy exactly, where it still cannot produce partial transmittance), and the difference it shows—a flame that sits in front of every primitive versus one that each primitive occludes according to its own depth—is the difference the architecture makes, measured where it can be measured honestly.

What a loaded scene costs. Table 4 times the fire alone. Rendering it *inside* a reconstruction is a separate, purely render-side cost, which we measure paired—fire alone, then the same fire with each scene loaded, in one session at 1280×800, three seeds each—across three of the four scenes of §5.5: the 45 808-splat synthetic test scene, the 729 826-Gaussian KITCHEN benchmark, and the 380k captured campsite. TREEHILL is prepared for the occlusion comparison of Figure 11 and is not in this timing set; at 2 074 009 splats it would extend the trend below rather than test it.

The simulation step is unchanged throughout (12.5 ms alone against 12.9, 13.0 and 11.5 ms with the three scenes loaded): the scene is static geometry that never enters the Biot–Savart sum. Gather-and-sort is likewise flat (2.1 → 1.7/2.0/2.0 ms). What grows is the draw, and only the draw: from 5.0 ms for the fire alone to 21.8 ms with the synthetic scene, 147.4 ms with the campsite and 277.3 ms with KITCHEN. End to end, the interactive preset goes from 19.7 ms (51 FPS) to 36.4 ms (28 FPS), 160.9 ms (6.2 FPS) and 292.4 ms (3.4 FPS). The fire-alone baseline in this session is 19.7 ms against the 17.1 ms of Table 4, measured on the same machine from a bit-identical simulation (the replay hashes match): that ∼15% is session-level drift in the browser, not a regression, and it is the reason the paired form matters—every marginal figure below is a subtraction *within* one session and is immune to it.

Two things follow. The cost is linear in splats drawn, and—once the fire's own 5.0 ms draw is subtracted—the marginal cost per scene Gaussian is flat: 365 ns at 45.8k, 375 ns at 380k, 373 ns at 729 826, a spread of 3% over a 16× range in scene size. An earlier version of this measurement, taken at a different output resolution and without subtracting the fire's own draw, suggested the rate fell with scene size, and a later one that it rose; measured paired and subtracted, it does neither. And it bounds the claim sharply: the fire is real-time, the fire composited with a multi-hundred-thousand-splat captured reconstruction at 1280×800 is not, and the entire

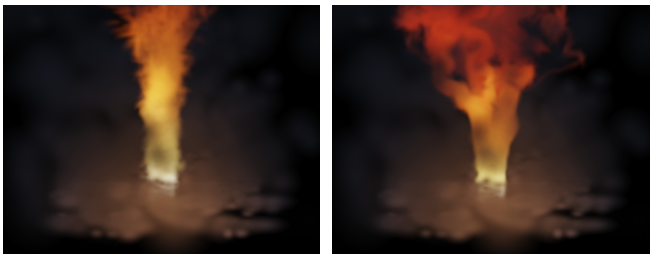


Fig. 9. Source hot-swap through the layer contract, same renderer and parameters. *Left:* all nine layers driven by the kinematic curl-noise plume—adequate bulk shape, but uniform ascent without vortical rolling. *Right:* all layers driven by the Gaussian vortex solver—baroclinic vorticity rolls the plume edges and produces the characteristic unsteady flame silhouette. Layers can be swapped per-layer at runtime with zero renderer changes. Each ablation figure uses its own seed (this pair, 42; covariance, 131; post, 257) so that three different comparisons do not show the same state three times; within a pair the seed, step count, camera and shading are identical, so the only variable is the named one.

deficit is rasterization—it scales with splats drawn, is independent of the solver, and is the part a production implementation would attack first (level-of-detail on the reconstruction, tile-based sorting, or simply a GPU port of the same passes, none of which touch the method).

Is our rasterization of the reconstruction faithful? Every scene-side claim above rests on our own renderer drawing someone else's reconstruction correctly, so we price that against a reference implementation and, through it, against the photographs. First, our WebGL2 rasterizer is set to a plain, unstyled mode—no bloom, no tone styling, no fire—and made to draw the prepared KITCHEN `.ply` at a fixed camera and 1400×980; `gsplat` 1.5.2 rasterizes the identical file at the identical camera, intrinsics and resolution. The two agree to 38.67 dB PSNR (mean absolute error 0.0051 in $[0, 1]$). This is an equality test between two forward rasterizers on one file, not a quality score: it says our EWA projection, depth ordering, spherical-harmonic evaluation and emission–absorption blend reproduce a reference CUDA implementation, and it is the measurement that caught five defects in our projection path, which stood at 14.97 dB before they were fixed. Second, to show the loader itself is not introducing the error, `gsplat` is driven through our own `.ply` parser against the real Mip-NeRF 360 photographs at the released COLMAP poses, on the *uncropped* 1 852 335-Gaussian release: 32.76 dB / SSIM 0.951 over 21 training views and 29.88 dB / SSIM 0.917 over 3 held out (every 11th registered image, standard 1:8 hold-out, half resolution). Those are in the neighbourhood of the published 3DGS figures for this scene, which is what makes the first number a comparison against something known-correct rather than a circular one. We claim nothing from this about the *prepared* asset's fidelity to the photographs—the crop deletes room content the photographs contain, and scoring it against them would price that deletion as reconstruction error.

Slice orientation. Orbiting the camera toward the slice planes' edge separates world-fixed slices into the flat sheets they are, and at 90° collapses them into lines, while view-aligned slices continue to read as a volume (Figure 12). We quantify this with an unclipped probe: total HDR energy in the framebuffer *before* tone mapping, swept over 36 azimuths at a fixed simulation state. World-fixed slices lose up to 89% of their radiant energy edge-on (min/max ratio 0.11 across the sweep, coefficient of variation 0.27); view-aligned slices are azimuth-invariant by construction, and the probe confirms it to numerical precision (CV $< 10^{-3}$). An earlier peak-brightness version of this probe saturated at the display ceiling and was withdrawn; measuring pre-tonemap energy removes the clipping.

Inter-slice coupling: a cautionary measurement. We attempted to quantify the effect of Eq. 8 with a snapshot probe—the mean Pearson correlation of adjacent slices' velocity fields. An informal sequential measurement (settings changed on a warm, running simulation)

Table 4. Per-frame cost (ms) vs. slice count N and particles per layer N_p, for both solvers at matched settings. Every row is the fire alone—no scene is loaded, and the 640 procedural environment splats of §5.6 are off, which is why the quality preset appears here as 11.6k Gaussians against the ∼12.3k quoted with them in §6; drawing the same fire inside a reconstruction raises the draw from 5.0 ms to 21.8 ms with the 45.8k synthetic scene, to 147.4 ms with the 380k campsite and to 277.3 ms with the 729 826-Gaussian KITCHEN benchmark, reported separately in §7 (*What a loaded scene costs*). sim_{VIC} = retained vortex-in-cell baseline; sim_{VPM} = grid-free Gaussian vortex solver at the accuracy setting θ=0.7 in every row *except* the last, which is the shipping interactive preset at θ=0.9—the same 9×600 state with the treecode opened up, which is why that one configuration appears twice at different costs; sort = gather + exact depth sort; render = GPU-synchronized draw + HDR post. Sort and render depend on the Gaussian population rather than on the solver that produced it; both were recorded in every run and the values printed are the grid-free run's. Across the nine matched configurations the vortex-in-cell run agrees to within seed noise (median ratio 0.90 for sort and 1.07 for render, against a median five-seed spread of 12–15%). The exception is the last N_p=2200 row, where the two runs did not reach the same population—19.6k Gaussians for the vortex-in-cell run against the 15.0k printed here—and it sorts in 9.86 ms rather than 6.54; per Gaussian the two agree to 14%. Total is therefore given for the grid-free solver only; the corresponding measured vortex-in-cell frame times are 2.44, 8.32, 14.05, 24.33, 34.86, 45.93 ms down the N_p=1300 block and 8.43, 15.01, 40.15 ms down the N=9 block. Every number is the median of five seeds, all timed in the same Chrome session (V8 JIT, ANGLE D3D11) in one scripted run; simulation is single-threaded CPU JavaScript; every row reproducible from a seed.

N	N_p	Gaussians	sim_{VIC}	sim_{VPM}	sort	render	$\text{total}_{\text{VPM}}$ (FPS)
1	1300	1.3k	1.32	3.1	0.40	0.58	4.1 (246)
3	1300	3.9k	4.08	9.6	1.40	4.98	16.0 (62)
5	1300	6.5k	6.91	15.5	2.17	4.95	22.6 (44)
9	**1300**	**11.6k**	**12.9**	**29.8**	**4.24**	**7.77**	**41.8 (24)**
13	1300	16.7k	18.2	42.9	7.00	10.4	60.3 (17)
17	1300	21.8k	23.6	60.5	10.9	13.7	85.0 (12)
9	300	2.7k	5.38	5.9	0.84	2.91	9.7 (103)
9	600	5.4k	7.85	13.1	2.06	5.55	20.7 (48)
9	2200	15.0k	19.7	42.0	6.54	9.37	57.9 (17)
Shipping interactive preset — the same 9×600 state, treecode opened to θ=0.9:							
9	600	5.4k	7.85	10.4	1.74	5.05	17.1 (58)

suggested a strong monotonic rise with κ_z; under the deterministic-replay protocol (per setting: 3 seeds × 4 snapshots after a 600-frame cold-start warm-up, c=0) that trend *does not replicate*: the probe sits at 0.26–0.38 across $\kappa_z \in [0, 1.2]$ with no significant trend, because the shared source geometry already induces strong structural correlation between slices and the probe's between-setting variance swamps the operator's marginal effect. We therefore make no coherence claim from *this probe*. The claim that survives it comes from the field-export protocol below, whose ensemble statistics are not dominated by the shared source geometry: there the particle-strength-exchange form of the operator raises adjacent-slice correlation from 0.18 to 0.62 (four-seed means). We report the failed probe deliberately: the replay harness exists so that informal numbers cannot survive into print, and the operator's grid-era version had passed exactly this kind of informal reading before failing the controlled one.

A true-3D solver through our own pipeline. The comparison we consider most informative is not against another system but against *ourselves without the layering*, because that is the only way to isolate the layered decomposition from every other difference. We therefore implemented a true 3D Gaussian vortex-particle solver inside the shipping prototype and rendered it through the identical path. Everything is held fixed: the world domain (the 3D vortons occupy exactly the slab the slice stack occupies, with the same out-of-slab clamp), the kernel family and core radius σ_0, the treecode opening angle θ, the particle budget, the baroclinic coefficient, the two-temperature scheme, buoyancy, cooling and Δt, the particle-to-Gaussian mapping, the depth sort, and the rasterizer. Two things differ, and they are what "3D" means: circulation is a three-vector $\boldsymbol{\alpha}_i$ inducing velocity through the 3D Gaussian-regularized Biot–Savart kernel (an erf-form closed expression [Winckelmans and Leonard 1993]), and particles carry the vortex-stretching term $(\boldsymbol{\alpha} \cdot \nabla)\mathbf{u}$—evaluated analytically from the same kernel's gradient—which the stack structurally cannot have. A retained 3D vortex-in-cell solver provides the corresponding grid-based control.

The cost of the layered decomposition is therefore measurable directly. At matched budget (~5.4k particles, θ=0.9 on both sides), the 3D solver's simulation step is 2.8× the stack's (27.5 vs. 9.9 ms; simulation-only timings, taken outside the browser session of Table 4—a five-seed re-measurement of the same code puts the ratio at 3.1×, so we quote the lower of the two); switching stretching off changes its cost by nothing measurable, so the ratio prices dimensionality, not the ablation. Gather-and-sort and render are unchanged, as they must be, since only the source of the particle state differs. Running the comparison the other way—holding *wall-clock* fixed instead of count, 215 vortons per slab sized from the measured 2.8× ratio at θ=0.9 (the spectral exports themselves run at θ=0.7, where the same sizing would give 176)—the 3D solver still reaches spectral distance 0.23 with ρ=0.77: a better in-plane spectrum than the shipping stack at the same coherence. What it cannot do at that budget is *be the picture*: 1.9k renderable Gaussians no longer tile the flame that the stack's 5.4k do, and the deficit is exactly the image-quality criterion that set N_p in §7. The stack's particle budget is a rendering constraint as much as a solver one; a system free to simulate on one set of elements and render on another would make the equal-cost 3D solve the better choice, and §8 lists that decoupling as future work.

We draw the honest conclusion. *The layered decomposition is not what makes this system real-time*—a matched 3D solve is under a factor of three away and remains interactive at the shipping budget. What layering buys is resolution allocation: in-plane detail is cheap and depth is charged only for the slices actually rendered, which is why the architecture scales to the slice counts production layered fire uses [Horvath and Geiger 2009]. What it costs is measured next, and having this solver in-process lets us ablate the term the stack cannot carry rather than argue about it (Figure 17).

Spectral comparison against a true-3D reference. Figure 18 confronts the layered approximation with a matched true-3D reference: a 32×48×32 Boussinesq Euler solve (semi-Lagrangian advection, Jacobi pressure projection, seeded symmetry-breaking forcing at the source; solver in supplemental) with the same domain, cell size, heat source, cooling, and Δt as the slice stack, compared on the same observable—the in-plane (u, v) fluctuation field on x–y planes sampled at the nine slices' exact world depths. The stack is exported from the shipping solver with procedural turbulence and wind disabled, isolating solver dynamics; both sides use the replay protocol (40 snapshots after warm-up, seeded). Three findings. *(i) The grid-free stack recovers most of the reference's spectral shape—a gap that was previously dominated by the grid.* The 3D reference distributes energy broadly (48% in the largest-scale bin; ~30% in the mid-band $k \in [2, 4]$ cycles/unit, tracking $k^{-5/3}$). The grid-free stack configurations sit at 50–67% largest-bin and 15–27% mid-band—log-spectral distance 0.39–0.68—where the retained vortex-in-cell stack, measured through the identical protocol on the same four seeds, concentrated 83–92% in the largest bin with only 2–5% mid-band (distance 0.78–0.87). Every figure on both sides is a per-configuration mean over seeds $\{7, 11, 17, 23\}$; the largest single-seed excursion from any mean is 0.035 in distance and 3.4 points in largest-bin share on the grid-free side, 0.075 and 2.4 points on the grid side, against a per-configuration gap of 0.19–0.38. The interpretation is direct: with the baroclinic source evaluated on the particles rather than through a 32×48 grid, the mid-band rolling that the grid's Nyquist suppressed is actually simulated. This is the measured payoff of grid freedom, purchased by the 2.3× simulation step of Table 4. Two caveats, on the observable and on the metric, stated once. The grid-free observable is a point-sampled continuous field, so a small tail of above-Nyquist kernel energy aliases into the export; at $\sigma_0 = 0.05$ the kernel retains ~4% amplitude at the sampling Nyquist, far too little to explain a mid-band that moved from 3% to 27%. And the distance weights all twelve bins equally, so 81–94% of its value on the grid-free side is set by the six bins above k=4.25 that together carry 2.3% of the reference's energy. Restricted to the six bins holding the other 97.7%—the band both lattices resolve—the distances become 0.05–0.26 for the grid-free stack against 0.66–0.87 for the grid-based one, a margin roughly four times wider than the all-bin figures report. We quote the all-bin metric throughout because it is the stricter one for us. Density remains the knob to check: the overlap condition $h \lesssim \sigma$ is satisfied at 600 particles per layer ($h = 0.018$ in the plume interior, $h/\sigma_0 = 0.37$), and repeating the entire measurement at 2600 per layer ($h/\sigma_0 = 0.25$) improves the distance moderately (0.46 → 0.38

mean log-spectral distance, a 17% reduction; best condition 0.31, largest bin 0.50 against the reference's 0.48, mid-band 0.28 against 0.30)—the result is near-converged in N_p, and refinement moves it *toward* the reference, not away.

Four further controls close the remaining confounds, and one of them cuts against us before it cuts for us. *(a) Grid refinement.* The obvious rejoinder—"give the grid more cells"—we test directly: at 64×96 per slice (iteration count scaled with resolution) the vortex-in-cell stack recovers most of the in-plane spectral shape, as doubling the Nyquist must (log-distance 0.19 uncoupled, 0.41 coupled; three seeds). That advantage, we should say, lives in the sparse high-k bins: over the band carrying 97.7% of the reference's energy the ordering reverses, the finer grid scoring 0.24 and 0.34 in the two configurations it was run in where the grid-free stack scores 0.16 and 0.13. What it does not recover, on either metric, is coherence: its grid-form coupling reaches only ρ=0.41 where the pairwise particle exchange reaches 0.76, and its cost, 33.4 ms per frame at 9×600, already exceeds the grid-free solver's 29.8 ms at 9×1300. *(b) The core radius, swept.* The grid-free solver's own resolution dial costs nothing to turn: at σ_0=0.035 (overlap ratio $h/\sigma_0 \approx 0.51$, still tiled; simulation cost unchanged) the spectral distance drops to 0.16 with ρ=0.61 retained. On the three axes together—spectral shape, z-coherence, milliseconds—no grid configuration we measured dominates this setting; the finer grid buys shape alone, at higher cost, with less than half the coherence. The shipping σ_0=0.05 is chosen for image quality (blob coverage), not spectral fidelity, and the dial is free to re-turn per shot; at $\sigma_0 \geq 0.07$ the field over-smooths rapidly (distance 1.5–2.3, adjacent slices nearly identical at $\rho \to 0.99$), so the usable band is real but narrow. That last correlation is not evidence that smoothing alone produces coherence: the sweep is run with the exchange operator on, and σ_0 also sets that operator's interaction radius, so widening the core widens the z-exchange itself. *(c) Smoothing alone, as a null.* Since *any* diffusive operator might be suspected of raising a correlation between neighbouring planes, we run the null directly: a separable in-plane Gaussian blur applied to the *uncoupled* stack, which cannot communicate across z at all. It moves ρ from 0.181 to at most 0.225—reached at a blur three times the width of the entire σ_0 sweep, after which ρ falls again—while taking the spectral distance from 0.53 to 3.82. The exchange operator buys +0.44 in ρ for +0.15 in distance; in-plane diffusion buys +0.04 for +3.3. Blurring the coupled stack likewise moves its ρ by under 0.01. The coherence is the operator's, not the smoothing's. *(d) Timestep.* Repeating the full measurement at $\Delta t = 1/30$ and $1/120$ s moves the distance by less than 0.03 in either direction (0.59/0.58/0.55)—the observable is timestep-converged.

The remaining confound is implementation: the reference is separate NumPy code. The in-prototype 3D vortex-particle control (§7) removes it—same kernel, same σ_0, same θ, same particle scheme, sampled through the same export protocol—and independently reproduces the external reference's character (largest bin 0.43 vs. 0.48; mid-band 0.34 vs. 0.30), corroborating both. It also prices the one term the stack structurally cannot have: *switching vortex stretching off inside the matched 3D solver moves its spectrum by only* 0.06 (largest bin 0.43 → 0.40, mid-band 0.34 → 0.37), against a stack-to-3D distance of 0.28–0.40. Under this discretization the missing stretching cascade explains only ∼20% of a much smaller gap; the rest is the loss of three-component transport. We note plainly that the corresponding grid-based control had attributed ∼70% of its (four times larger) gap to stretching: the attribution is discretization-dependent, and the grid-free pair is the better-matched control for the shipping solver. *(ii) The two couplings now trade off measurably* (four-seed means): coarse coupling improves spectral shape (distance 0.53 uncoupled → 0.39 at c=0.5), while the inter-slice exchange slightly smooths it (0.53 → 0.68 at κ_z=0.45; 0.39 → 0.54 with both) —a diffusion, as its construction says it is—and buys coherence instead. *(iii) z-coherence, the stack's clearest deficit, is now largely recoverable.* Adjacent reference planes correlate at $\rho \approx 0.93$ and the matched 3D control at 0.80; the uncoupled stack sits at 0.18 (four-seed mean; range 0.11–0.22). Coarse coupling raises it to 0.29 (0.19–0.38, the most seed-sensitive number we report); the particle-strength-exchange operator of Eq. 8 raises it to 0.62 alone and 0.76 combined (range 0.66–0.82; $\rho(\Delta z)$ RMS distance to the reference 0.15)—where the same operator in grid form had measured an inconclusive 0.13–0.19 at 32×48 and reaches only 0.41 even at 64×96. The pairwise exchange acts on the circulations the induced field is actually made of, rather than on a resampled vorticity grid, and the coherence it produces survives the Biot–Savart reconstruction. The spectral price in (ii) is the flip side; the two axes must be read together, and neither implies the other. The coherence is also directly visible: Figure 19 shows the same seed at a 63° world-fixed side view with the operator off and on—independent per-slice flames, each with its own hot core, fuse into a single connected volume. These curves make §8's honesty quantitative—and they are the explicit target any future inter-slice coupling must be measured against. The reference itself is a modest solve (32×48×32, Jacobi projection); we use it as a matched control, not as a converged ground truth, and say so.

Scene occlusion. Figure 10 exercises the compositing claim directly, against a ladder of three baselines so that no panel is broken for an unrelated reason. The figure's camera is chosen by measurement rather than by eye, because on a reconstruction the viewpoint decides what is being measured. It is a training view of the capture itself, raised 0.20 units, at which the prepared asset covers the frame: mean rendered alpha 0.9885 and 0.004% of the frame pure black, against 0.9929 and 0.0004% for the uncropped release at the same camera. The viewpoint used in an earlier draft sat 3.17 units from the nearest training camera—14× the 0.22 median inter-camera spacing, outside the training cameras' own shell—and produced a thin-streak statistic of 0.70 per mille against a training-set maximum of 0.35. Those streaks are extrapolation artifacts of the reconstruction, not of our renderer: they appear identically in the authors' released checkpoint rendered by a reference implementation. A compositing comparison made there would be measuring the extrapolation. The *depth-aware* baseline (top right) is the strongest alternative we know how to build without interleaving: scene splats whose footprint opacity exceeds a threshold write a depth buffer and fire fragments are z-tested against it, the natural upgrade of backdrop compositing [Shen et al. 2026] to per-pixel occlusion. On this capture it succeeds, and the honest report is that it ties with us. The threshold is swept rather than chosen and the sweep is nearly flat—99.9, 100.1, 100.4, 101.4 and 110.2% of our fire contribution at

0.15, 0.30, 0.50, 0.70 and 0.90—with the drift at the top end being the baseline shedding real occluders and approaching the backdrop composite. At 0.50 the two panels differ in 3 134 of 1 372 000 pixels. (Three appearance settings do change with the viewpoint—emissive gain 2.2 → 3.0, bloom strength 1 → 1.4, and the HDR bright-pass threshold 0.40 → 1.00, near the demo's own 0.95 default—because a frame filled with a brightly-lit room blooms at 0.40 where the earlier, mostly-empty frame did not; at 1.00 the bloom is 97% fire, at 0.40 it was 95% scene. No simulation parameter changes, and all four panels share one setting throughout.) The explanation is the occluder: the model loader's lift arms are opaque, so 85.5% of the flame's silhouette is fully transmitted and 12.0% fully blocked with only 2.5% partial, and binary occlusion is the case a z-test handles exactly. The structural argument of §5.6—that no single depth per pixel exists across interpenetrating semi-transparent primitives—is therefore not tested by this scene, which puts nothing translucent in front of a table-top fire; it is tested in the next paragraph, on a capture that does. What this figure still separates is our composition from the prior art's: the *backdrop* baseline (bottom left) removes the depth test and puts the flame's white-hot base in front of the lift arms that should hide it (119.3%), and the *unsorted* panel (bottom right) additionally drops the intra-scene sort (112.8%), which we report separately rather than let that defect stand in for the prior art. The interleave's overhead is now measured at both scene sizes rather than one: 22.9 vs. 21.2 ms median render time at 45.8k synthetic scene splats under the 9×1300 flame (+8%), and 291.0 vs. 239.3 ms at the 729 826 splats of KITCHEN under the shipping preset (+22%); per scene splat, 38 against 71 ns, so the premium grows with scene size (same session as the paired timings above).

A semi-transparent occluder. Figure 11 runs the same ladder against foliage, and the result is different in kind. The scene is TREEHILL, a second Mip-NeRF 360 release, at its own training camera `_DSC8979` (§5.5); the choice was made by search rather than by eye, over all eight pre-trained scenes at their registered cameras, ranked on the fraction of a candidate flame's silhouette whose transmittance through the intervening reconstruction lies strictly inside $(0.05, 0.95)$. Simulation state is bit-identical across every panel (replay hash `de38f81b`, confirmed four times) and the only settings that differ from Figure 10 are the three appearance values a brighter scene forces—emissive gain 3.0 → 4.0, bright-pass threshold 1.00 → 1.20, bloom unchanged at 1.4—chosen so that the bloom halo does not smear the veil the figure exists to show. Transmittance is measured in the renderer on its linear path by differencing two emissive gains, which cancels the scene term exactly; the backdrop composite, which draws the fire unoccluded, is the reference and returns $T = 1.0000$ over 100.0% of the silhouette. Over the 50 355-pixel silhouette our composition transmits partially at 88.1% of pixels (mean $T = 0.616$, and a continuous histogram), while the depth-aware baseline is bimodal at every threshold: 9.3, 8.4, 5.1, 2.8 and 0.6% partial at $\alpha = 0.15, 0.30, 0.50, 0.70, 0.90$. The threshold sweep of the fire's contribution is no longer flat—83.7, 108.6, 130.1, 141.9 and 149.4% at those five settings—so unlike KITCHEN there is a threshold that matches us on energy, and we shot it rather than report around it: at $\alpha = 0.22$ the baseline is at 97.7% and at 0.25 it is at 102.0%. At that matched point it is still wrong everywhere it matters. Partial transmittance is 9.1% against our 88.1%; the silhouette splits into 51.5% fully open and 39.4% fully blocked; silhouette MAE is 16.5/255 and 92.0% of silhouette pixels differ from ours by ≥3/255, against 3 134 of 1 372 000 pixels differing on KITCHEN. The white-hot core saturates over 14.3% of the silhouette, and excluding every clipped pixel strengthens rather than weakens the result (88.1 → 96.6% partial for ours, 6.0–10.6% for the baselines); sweeping the silhouette threshold from 3 to 40 leaves ours at 82.0–88.1% against 2.8–5.1% at $\alpha = 0.50$ throughout. An offline instrument run on the raw `.ply`, wholly outside the renderer, independently predicts 93.9% partial at mean $T = 0.713$. Taken with KITCHEN's tie, this is the shape of the finding we would want a reader to carry away: the two compositions are indistinguishable on opaque occluders, and the interleave's advantage is exactly as large as the occluder is semi-transparent.

Ablations. Figures 9–16 ablate the major components: simulated vs. procedural sources, coupling strength, covariance dynamics, the layered representation itself, and HDR post. Each toggle is live in the interactive prototype.

Interactive control. Because dynamics are forward-simulated, the system responds immediately to user input: shift-dragging moves the fire source and the plume follows with vortical lag; wind tilts the column; the coupling, buoyancy, baroclinic, and cooling gains are all live sliders; and fire and smoke are two shading presets over the same particle state, switchable per keystroke.



Fig. 10. Per-primitive occlusion inside a public benchmark reconstruction: KITCHEN from Mip-NeRF 360, as released pre-trained by the authors of 3D Gaussian splatting [Kerbl et al. 2023] and prepared for our renderer by the documented offline step (§5.5)—729 826 Gaussians at degree-3 spherical harmonics. A small fire burns on the table behind the model Technic loader, its base crossing the machine's lift arms and black hydraulic linkage and its plume rising in front of the shelf beyond. The camera is a training view of the capture, raised 0.20 units, so it sits inside the reconstruction's own view distribution rather than extrapolating outside it (§7). Identical simulation state, camera and shading in all four panels; only the composition differs, and no simulation parameter differs from the shipping preset. We report each composition's *fire contribution*: the summed absolute RGB difference between the frame and an otherwise identical frame with the simulation reset and no particles emitted. This isolates what the fire adds without depending on its colour—a threshold on warm, saturated pixels looks reasonable but counts the scene's own red and wooden surfaces once the reconstruction is rendered at full saturation, which is how we mismeasured this comparison at first. *Top left (ours):* bucketed depth interleaving (§5.6); the lift arms cut the white-hot base away and the flame reads as being in the room rather than on it (the reference, 100%). *Top right (depth-aware baseline):* the strongest non-interleaved composition we know how to build—scene splats whose footprint opacity exceeds a threshold write depth, and fire fragments are z-tested against it before compositing over the scene. On this occluder it is *right* and we report that: 100.4% of the fire, differing from our panel in 3 134 of 1 372 000 pixels. The threshold is swept rather than chosen and barely matters—99.9, 100.1, 100.4, 101.4, 110.2% at 0.15, 0.30, 0.50, 0.70, 0.90—drifting toward the backdrop composite only once it starts excluding real surfaces. The occluder is opaque plastic: over the flame's silhouette 85.5% is fully transmitted and 12.0% fully blocked, only 2.5% partial, and binary occlusion is exactly what a z-test expresses correctly. What per-pixel depth cannot express is partial transmittance through interpenetrating semi-transparent primitives, which this capture does not put in front of the fire (§5.5); that case is measured on a second benchmark reconstruction in Figure 11, and this panel is the control for it. *Bottom left (backdrop baseline):* the scene is sorted *correctly* among itself but composites as a whole behind the flame—the analogue of compositing a ray-marched volume onto a reconstructed backdrop [Shen et al. 2026]. The fire is 119.3% of its correct contribution, and the visible difference is the whole white-hot base, which should be behind the lift arms and instead sits in front of them. *Bottom right (unsorted):* additionally dropping the intra-scene sort; the reconstruction dissolves and the flame floats over it at 112.8%. The depth-aware panel is the fair comparison, and on this scene it is a tie. The same four compositions on the redistributable synthetic scene are in `supplemental/figures-synthetic-occlusion/`, where a fallen log lies across the front of the fire. That comparison agrees with this one and for the same reason: ours 100%, the depth-aware sweep 101.8–107.9% across the same five thresholds, backdrop 117.9%, unsorted 121.7%; over the flame's silhouette 86.0% is fully transmitted and 11.8% fully blocked with 2.2% partial—every one of that scene's 45 808 splats is authored at opacity 0.950–0.970, so it contains no semi-transparent material either. Read together with Figure 11, which is the same comparison against foliage, this tie is the informative half of the result: our composition and a per-pixel z-test agree wherever the occluder is binary, and separate only where it is not.

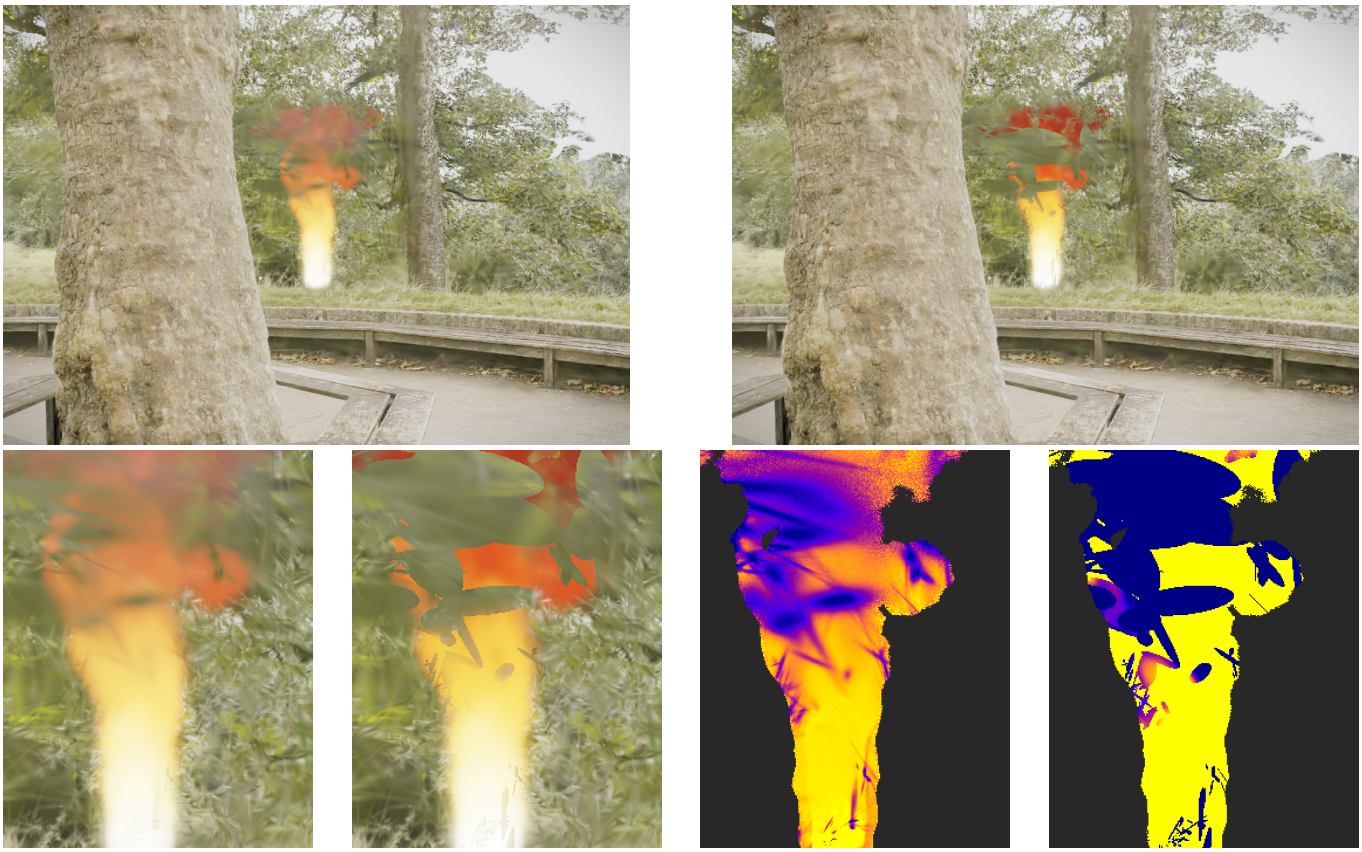


Fig. 11. The comparison Figure 10 could not make: the same two compositions against an occluder that is actually semi-transparent. The scene is TREEHILL from Mip-NeRF 360 [Barron et al. 2022], in the pre-trained release of [Kerbl et al. 2023] and prepared by the documented offline step (§5.5)—2 074 009 Gaussians at degree-3 spherical harmonics above an opacity floor of 0.005, which is low because the foliage veil *is* the low-opacity Gaussians. The viewpoint is the capture's own training camera _DSC8979: the eye sits exactly at that camera's centre, re-aimed onto the fire by 9.99° and narrowed in field of view, the smallest re-aim of any TREEHILL camera that does not put the flame behind the plane tree's trunk. A fire burns on the ground behind the low park wall, in a pocket of clipped hedge; the trunk stands immediately to its left as a *hard opaque* occluder in the same frame, so one image carries both regimes. Simulation state is bit-identical in every panel (seed 42, six steps, replay hash de38f81b verified four times) and no simulation parameter differs from the shipping preset; only the composition changes. *Top left (ours):* bucketed depth interleaving. *Top right:* the depth-aware baseline—scene splats above an opacity threshold write depth, fire fragments are z-tested against it—at α=0.22, the threshold at which it *matches* our total fire contribution, so that the comparison cannot be won by a gain. *Bottom, left pair:* 3× crops of those two panels on the flame. *Bottom, right pair:* measured per-pixel transmittance through the scene over the same crop, ours then the baseline, black outside the flame's silhouette. Transmittance is read out of the renderer itself on its linear path: fire radiance is linear in the emissive gain while fire alpha is not, so differencing two gains cancels the scene term exactly, and the backdrop composite—which draws the fire first into an empty buffer—supplies the unoccluded reference. That reference returns $T = 1.0000$ over 100.0% of the silhouette, as it must. Over the 50 355-pixel silhouette (3.67% of the frame), our transmittance histogram is continuous—3.1, 6.2, 8.3, 7.4, 8.1, 10.3, 10.5, 13.7, 13.2, 19.4% in ten equal bins, mean 0.616, 88.1% of pixels strictly partial. The energy-matched baseline's is two spikes: 51.5% fully open, 39.4% fully blocked, 9.1% partial, and it arrives at a similar mean (0.561) by averaging a binary cut rather than by transmitting partially anywhere. Visually that cut is the hard-edged opaque ellipses in the crop—individual scene Gaussians that happen to clear the threshold, written as though they were surfaces—against the lace of grass and leaf edges with haze between them in ours. It is not a tuning failure: 92.0% of silhouette pixels differ from ours by $\geq 3/255$ at the matched threshold (silhouette MAE 16.5/255), and 91–95% at every threshold in the sweep. We report the sweep and its crossing rather than a chosen point: as a fraction of our fire contribution over the silhouette the baseline gives 83.7, 94.6, 97.7, 102.0, 106.1, 108.6, 130.1, 141.9 and 149.4% at $\alpha = 0.15, 0.20, 0.22, 0.25, 0.28, 0.30, 0.50, 0.70, 0.90$, crossing 100% near 0.235, with backdrop at 150.9% and unsorted at 153.0%; the aggregate can therefore be matched, and the partial-transmittance structure cannot be, which is exactly the structural claim of §5.6. Two robustness checks, because the white-hot core saturates: 14.3% of the silhouette clips at 255, and discarding every clipped pixel moves our partial fraction 88.1 → 96.6% while the baselines stay at 6.0–10.6%, so the clipping works against us rather than for us; and an independent offline instrument, run on the raw `.ply` outside the renderer, predicts 93.9% partial at mean transmittance 0.713 against the renderer's 88.1% and 0.616—the same phenomenon, the residual gap expected because that probe places the flame on a single median depth plane while the renderer integrates its real extent. Against the opaque Lego occluder of Figure 10 the same baseline scored 99.9–110.2% with 2.5% partial and tied with us. The pair is the result: per-primitive interleaving and a per-pixel z-test agree on binary occluders and diverge on semi-transparent ones, which is what the architecture predicts and what neither figure alone would show. The remaining thresholds, the backdrop and unsorted panels, the full-frame transmittance maps and the exact operation sequence are in the supplemental.

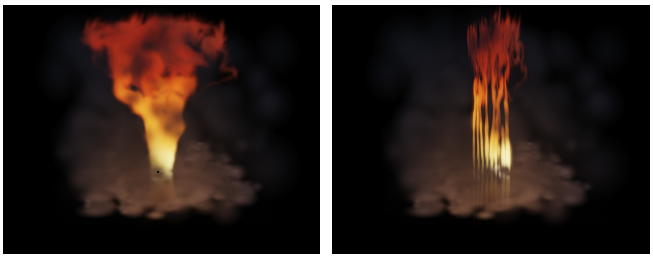


Fig. 12. Slice orientation. Same simulation state, seed, camera and shading; only the slice frame differs. The camera sits at a 79° orbit—close enough to the slice planes' edge to expose the degeneracy, oblique enough that the individual planes are still distinguishable. *Left:* view-aligned slices re-orient with the camera, so the stack integrates into a volume and reads correctly from any azimuth. *Right:* world-fixed slices seen near edge-on separate into the flat sheets they actually are—the plume loses its silhouette and becomes nine ribbons standing on the ground. At exactly 90° they collapse further, into single-pixel lines. This is the failure mode that motivates view alignment, and the reason a Lagrangian state that can be re-sliced per frame at no re-simulation cost is worth having.



Fig. 13. Inter-layer coupling (Eq. 7). *Left:* c=0, independent slices—the stack reads as several superimposed fires with uncorrelated sway. *Right:* c=1, slices slaved to the coarse solver—one coherent but overly smooth column. The default c=0.5 (Figure 1) preserves per-slice turbulence on a shared bulk motion. Snapshot field-correlation probes proved statistically unstable for quantifying either coupling mechanism (§7); the comparison here is deliberately qualitative.



Fig. 14. Covariance dynamics. *Left:* isotropic splats (k_Σ=0) read as soft blobs. *Middle:* flow-driven covariance (Eq. 9) shears splats into flame licks. *Right:* adding the trajectory filament stretch (Eq. 10) elongates fast particles into path-aligned streaks.

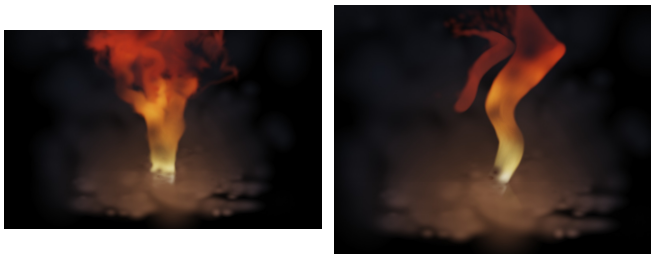


Fig. 15. Layered 2.5D vs. a matched-budget kinematic 3D mode. *Left:* nine coupled 2D vortex-particle slices (default). *Right:* the same total particle budget advected by analytic 3D curl noise with entrainment—a volumetric alternative that lacks the baroclinic rolling produced by the slice solvers. The *quantitative* comparison against a genuine 3D reference solve is Figure 18; this panel is the qualitative matched-budget rendering ablation.



Fig. 16. HDR post ablation: without (left) and with (right) bloom before ACES tone mapping. The prefilter threshold admits only the emission-dominated core, so bloom is deliberately a small, local effect—it changes 1.5% of pixels—adding the incandescent halo around the white-hot base without lifting the plume as a whole. The threshold is a real dial, not decoration: set above the scene's peak HDR value it admits nothing and bloom becomes a no-op.

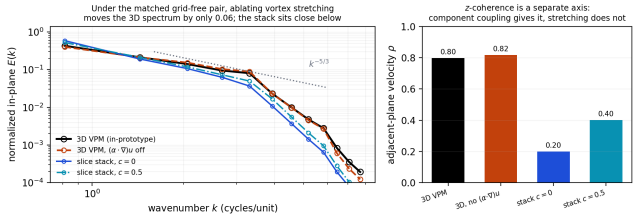


Fig. 17. Ablating the layered decomposition's missing term inside an otherwise identical solver (§7). The 3D Gaussian vortex-particle solver runs in the shipping prototype and shares the kernel family, core radius, opening angle, particle scheme, source terms, timestep, mapping and renderer with the slice stack, and is sampled on the same nine planes through the same protocol. *Left:* under the grid-free pair the stack's spectral gap to matched 3D is modest (log-L_1 0.28–0.40), and switching off vortex stretching $(\boldsymbol{\alpha} \cdot \nabla)\mathbf{u}$ inside the 3D solver moves its spectrum by only 0.06—under this discretization, most of the residual gap is the loss of three-component transport coupling, *not* the stretching cascade. (The retained grid-based pair had attributed ∼70% of a far larger gap to stretching: the attribution is discretization-dependent, which is itself a finding.) *Right:* z-coherence remains the stack's clearest deficit: adjacent-plane $\rho = 0.80$ for 3D against 0.20 (uncoupled) / 0.40 (coarse-coupled) for the stack; the inter-slice exchange of Eq. 8 recovers most of it (§7). Any future inter-slice operator must be measured on both axes.

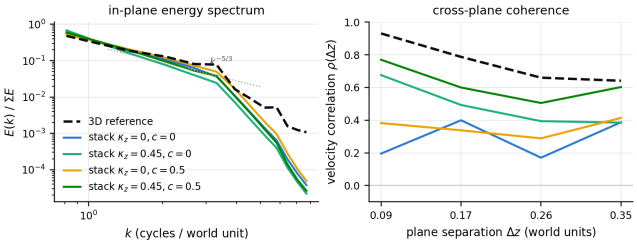


Fig. 18. The layered approximation, measured against a matched true-3D Boussinesq reference (dashed). *Left:* normalized in-plane energy spectra. The 3D reference spreads energy across scales ($\sim k^{-5/3}$ through the mid-band); the grid-free stack now tracks it through the mid-band (largest bin 0.50–0.67 across configurations, four-seed means, vs. 0.48; coarse coupling c sharpens the spectrum, the inter-slice exchange κ_z smooths it), a regime the retained grid-based stack never reached (0.83–0.92 largest-bin, same four seeds). Curves are exported at 600 particles per layer; repeating the measurement at 2600 moves them moderately *toward* the reference (§7). *Right:* cross-plane velocity correlation at the slices' depths. The 3D flow is strongly z-contiguous ($\rho \approx 0.93$ adjacent); the uncoupled stack is not (0.18), and the particle-strength-exchange operator recovers most of the gap (0.62 alone, 0.76 with c; four-seed means—the curves shown are the seed-7 member). Correlations shown for $\Delta z \leq 0.35$; larger separations in the reference are contaminated by the source's mirror symmetry (supplemental).

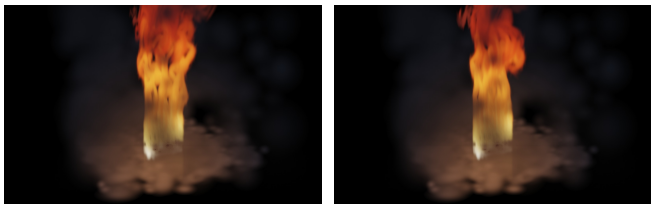


Fig. 19. The inter-slice exchange operator, seen rather than plotted: same seed, same step count, world-fixed slices from a 63° side view (turbulence, wind and coarse coupling disabled to isolate the operator). *Left* (κ_z=0): each slice burns as an independent flame with its own hot core—the stack reads as separate sheets. *Right* (κ_z=0.45): the pairwise circulation exchange fuses the bases into one core and the bodies into one connected volume, the visual counterpart of ρ: 0.18 → 0.62 (§7).

8 Limitations and Future Work

Physics of stacked 2D slices. Vortex stretching $(\boldsymbol{\omega}\cdot\nabla)\mathbf{u}$ vanishes in 2D, and 2D turbulence cascades energy toward *large* scales, opposite to 3D; a slice stack therefore cannot reproduce the statistics of three-dimensional fire turbulence, however plausible it looks. We have *measured* what remains of this gap under the grid-free solver (Figure 18): the stack holds 50–67% of in-plane energy in the largest-scale bin where a matched 3D reference holds 48% (log-spectral distance 0.28–0.40 to the matched in-prototype control), and uncoupled adjacent-plane coherence is 0.18 vs. the reference's 0.93—recoverable to 0.76 by the inter-slice exchange (Eq. 8), which reintroduces the dropped out-of-plane *transport* at a measured spectral-smoothing price. What no exchange supplies is vortex-stretching *amplification*; under the matched grid-free control its ablation prices at ∼20% of the residual spectral gap, the rest being three-component transport itself. We deliberately trade the remaining fidelity for interactivity in a regime—turbulent, shapeless, emission-dominated media—where the eye is forgiving: visual plausibility rests on the large and mid scales the grid-free stack does capture plus appearance-side detail (FBM footprints, flow-driven covariance, bloom), not on reproducing full 3D statistics. Figure 18 is the standing target for any future coupling design.

Illumination. Fire-to-scene lighting is a screen-space approximation, not transport: Gaussian scene splats are warmed by proximity to the flame's image-space position. This is the one axis on which the closest prior system is straightforwardly ahead of us and we should say so: FieryGS treats simulation voxels above a temperature threshold as volumetric light sources and adds a diffuse and specular term per Gaussian, a world-space shading term where ours is an image-space one, and it can do this because it has already paid for a 256^3 grid and a ray march [Shen et al. 2026]. Correct volumetric emission from Gaussian mixtures is available in a ray-traced formulation [Condor et al. 2025]; a hybrid that shades nearby scene splats from the flame's particle set (a few thousand point lights, amenable to clustering) is the same idea at our representation's price, and is future work, as are cast shadows and smoke self-shadowing.

Renderer completeness. Our rasterizer implements the forward 3DGS math (sort, EWA projection, footprint, emission–absorption blend; spherical harmonics up to degree 3 for loaded scenes) but not the differentiable backward pass—our demo has no reconstruction objective—and fire splats themselves carry no view-dependent color, which isotropic incandescence does not need. On the campsite capture of §5.5 we nevertheless truncate to degree 1, and that is a loss we impose rather than a formality: that reconstruction is trained at degree 3, and we render it with its degree-2 and degree-3 bands truncated away and an opacity floor applied—a reduced form of the reconstruction, not the reconstruction as trained. The truncation is a budget decision at 3×10^6 input Gaussians, not a shader limit; KITCHEN is rendered at the full degree 3 it was trained at. The FBM modulation of the splat footprint also departs from the pure Gaussian kernel, so fire splats exported to a stock 3DGS viewer would render softer than shown here; an export-compatibility experiment would sharpen the GS-native claim. The exact CPU sort of the dynamic set is adequate at 10^4 Gaussians, and the $O(n{+}K)$ scene bucketing (§5.6) no longer has to be extrapolated to 10^5: a composited frame is measured at the 729 826 scene splats of the KITCHEN benchmark, where the whole of the added cost falls in the draw (272 ms at 1280×800, §7). Ordering between the streams is nonetheless bucket-granular, and what leaves the real-time budget at that splat count is rasterization—which is why tile-binned GPU sorting, and a GPU port of the draw it belongs to, remains the right implementation for dense fire inside full reconstructed scenes.

View dependence and multi-view consistency. View-aligned slices make the fire's world-space geometry a function of the rendering camera: the same particle lifts to different world positions as the camera orbits, so two simultaneous viewers—or the two eyes of a stereo pair—observe slightly different fires. This is the sharpest tension in the "GS-native" framing: the fire's *attributes* are camera-independent, but its slice lift is view-adapted, unlike a static reconstruction. (Solver-side grid freedom does not dissolve this tension—representation and view dependence are orthogonal axes, and we claim progress only on the former.) The world-fixed mode removes the dependence at the cost of edge-on collapse (Figure 12); for stereo, sharing one lift per eye pair is a cheap fix. More fundamentally, because particles are world-space objects, slices can be re-binned to a new axis without restarting the simulation—a remedy unavailable to raster-state slice methods, which we have not yet implemented. Very fast camera motion likewise changes the decomposition axis mid-shot, and flows predominantly along the view axis remain the weakest case [Horvath and Geiger 2009].

Radiometric level of detail. In the optically-thin, emission-dominated regime the front-to-back accumulation of Eq. 11 is approximately additive, so distant fire clusters could be merged into fewer Gaussians while conserving the summed emission $c\,\alpha$ and its emission-weighted second moment—a radiance-conserving LOD criterion, distinct from the photometric-loss LOD of opaque-splat hierarchies [Kerbl et al. 2024]. This is the concrete route to dense fire inside large reconstructions; we have not implemented it. The scene side already shows what its absence costs: the campsite capture of §5.5 is reduced from 2 998 953 Gaussians to 380k by a seeded *uniform* draw (after an opacity floor and a crop), which thins dense and sparse regions alike and conserves neither the summed $c\,\alpha$ nor its second moment. That the result looks acceptable is a property of this capture's density rather than evidence that uniform subsampling is the right reduction; a radiance-conserving criterion would spend the same budget where the reconstruction actually carries energy.

Geometry-aware flow. The loaded scene could also act on the *simulation*, not only the compositing: sampling the reconstruction's surfaces into boundary vortex Gaussians whose strengths cancel the normal flow—a panel method, or the method of images our ground mirror already instantiates (§4.2)—would deflect the flame around the captured geometry in real time, staying grid-free: the per-slice, Lagrangian counterpart of the occupancy-grid obstacle boundaries that offline scene-combustion imposes [Shen et al. 2026]. The captured scene makes the gap concrete rather than hypothetical: in Figure 10 the plume rises past the loader's lift arms without feeling

them—the machine occludes the flame correctly and does not deflect it at all.

Toward reconstruction. The mapping of Table 2 is differentiable end-to-end in principle. Because the forward model exposes only a handful of interpretable controls (source, wind, buoyancy, baroclinic and coupling gains, cooling), fitting *those* to a video summary statistic—a far lower-dimensional, better-constrained inverse than fitting a free field or free Gaussians [Nazarenus et al. 2025; Yu et al. 2023], and solvable even with gradient-free system identification [Jatavallabhula et al. 2021]—would recover not a frozen radiance field but a *running, editable* simulation. Demonstrating a single fitted example is the natural next step.

What this evaluation does not yet cover. We state the gaps explicitly rather than leave them to be discovered. *(i) Real captures.* The scene-side results run on a real trained reconstruction we neither captured nor trained (§5.5): compositing and per-primitive occlusion in Figure 10, the combustion loop in Figure 4, and the loaded-scene timings of §7. Two things are worth saying about what that changed. First, our earlier expectation was wrong, and then our correction to it was wrong as well: from a viewpoint outside the capture's own camera distribution the depth-aware baseline appeared to fail catastrophically on real geometry, and we wrote that up as a finding. Measured again from a training viewpoint, on a prepared asset that fills the frame, it does not fail at all—the occluders KITCHEN puts in front of a table-top fire are opaque, and a z-test resolves opaque occluders correctly. The lesson we would pass on is about the viewpoint, not the baseline: a compositing comparison rendered where the reconstruction was never observed measures the extrapolation, not the composition. Where the baseline does fail is where the structural argument said it would, and finding that case took a search rather than a viewpoint: over eight pre-trained benchmark reconstructions at their own registered cameras, ranked on measured partial transmittance, one capture puts genuine foliage in front of a fire, and there the two compositions separate at every threshold (Figure 11). Second, the splat adjacency graph really is stressed by a capture in the way we expected: fuel is a per-splat quantity and a reconstruction's density is neither uniform nor semantic, so which primitives carry fuel becomes a choice the system cannot make for itself (§5.7). What we cannot do is redistribute the capture—it is not ours—so the residual gap is reproducibility rather than realism. The same comparisons on a public benchmark reconstruction (Mip-NeRF 360, Tanks and Temples) are what we would add next; the `.ply` path that consumes them is the stock Inria format, unmodified, and the offline preparation is a shipped script. *(i b) Solver refinement.* The particle-density, core-radius, grid-resolution and timestep confounds on the spectral result are all now swept directly (§7), as is the treecode opening angle θ against the exact sum; the remaining unswept axis is hardware—all timings come from one workstation, and a second, weaker machine (integrated GPU) is the measurement we would add first. *(ii) Baselines.* The comparison we consider most informative—a true 3D solver rendered through *our own* pipeline, isolating the layered decomposition from every other difference—is now implemented and reported (§7). The compositing baselines are reported at swept rather than chosen settings: the depth-aware baseline's one free parameter is swept on both captures (§7, Figures 10 and 11), and on KITCHEN it *matches* us, which we report as the result rather than reaching for a viewpoint that would not. The comparison that separates the two—a semi-transparent occluder in front of the flame—was the single most valuable scene-side measurement missing from earlier versions of this paper, and it is now made: on TREEHILL the baseline can be handed the threshold that matches our fire energy and still produces 9.1% partial transmittance against our 88.1%. What remains open on this axis is narrower and worth naming: the measurement rests on one capture and one viewpoint, the opacity floor that preserves the veil (0.005) is far below the one we would ship for appearance, and we have not measured how the separation degrades as a reconstruction is thinned toward an interactive budget. We still report no head-to-head against another *system*: the published numbers we cite were measured on different hardware, at different resolutions, and with far larger simulation state, so we use them to place the design point rather than to claim a speedup. *(iii) Hardware.* All timings come from a single workstation; a low-end second point would bound the interactive claim. *(iv) Ablations.* A slice-count sweep at fixed total particle budget, and a non-saturating viewpoint-isotropy measure swept densely in azimuth, would replace the qualitative arguments in §4.3. *(v) Perceptual evidence.* Our central trade—physical fidelity for interactivity in a regime where the eye is forgiving—is an empirical claim about viewers that we argue for but *have not tested*: no participant has run our study. What we ship instead is the instrument, pre-registered so that its result cannot be shaped after the fact. It is a two-alternative forced choice over five of the ablations reported above (grid-free vs. grid solver at two seeds, simulated vs. procedural source, inter-slice exchange on/off, coarse coupling on/off, bloom on/off), 18 trials per participant, sides counterbalanced per trial and order shuffled, with 4 s looping *motion* clips rather than stills—fire is a temporal phenomenon and a still cannot test a solver difference. Both members of every pair are deterministic renders sharing seed, timestep, camera and shading, so the only difference is the named variable. The analysis is written and committed before any data exists: pooled two-sided exact binomial tests against chance with Wilson intervals, no outlier rejection and no response-time filtering, plus a per-participant direction count so a pooled effect driven by a minority is visible. The pre-registered target is $N \geq 12$. Every pair in it is fire-only; the compositing comparison is deliberately not among them, because its claim is structural rather than perceptual—per-pixel depth cannot express mutual occlusion among semi-transparent, interpenetrating primitives at all (§5.6)—and a preference test would not settle a structural question in any case. Figure 10 separates our composition from the backdrop model on real geometry and Figure 11 separates it from the depth-aware one; both are settled by measured transmittance, which is what a structural claim wants, and neither would have been settled by a preference test. We report this gap as a gap: the honest statement today is that the perceptual claim is untested, and the apparatus for testing it—stimuli, protocol, analysis—is part of the release.

9 Conclusion

We have presented a system in which fire and smoke live natively inside Gaussian-splatting scenes: not reconstructed, not ray-marched

over a backdrop, but forward-simulated in real time by vortex particles that *are* the rendered Gaussians—now in the strict sense: the solver evaluates its own velocity, gradient, and temperature fields in closed form from the same means and amplitudes the rasterizer draws, and deforms the covariances it draws with the gradient of its own velocity field, with no grid anywhere. Its most distinctive behavior is a closed loop—the flame ignites the reconstruction, combustion state lives on the scene's own splats, and burning geometry re-seeds the solver, so that what burns next is decided by the flow and not only by the adjacency structure—demonstrated on two captured reconstructions, though at these splat densities the loop runs six to nine times slower than real time (§5.7). The enabling observation, that Lagrangian vortex elements and 3D Gaussians are the same class of point primitive, collapses the simulation-to-rendering pipeline to a set of closed-form functionals of one Gaussian population; a solver whose entire state is the splats themselves—no pressure solve, no linear solve, no field to resample—is what keeps that loop inside an interactive frame budget. We have also measured what the layered decomposition costs, against a true 3D solver inside the same prototype and behind the same renderer, and used it to ablate the term the layering cannot carry rather than argue about it. Having demonstrated the pattern across flame, smoke, and embers, we conjecture that the same particle-set unification extends to other optically thin, surfaceless Lagrangian phenomena (sparks, spray, dust, aurorae), and that GS-native effects are a necessary ingredient if reconstructed scenes are to become fully dynamic worlds.

References

Alexis Angelidis and Fabrice Neyret. 2005. Simulation of Smoke Based on Vortex Filament Primitives. In *Proceedings of the 2005 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '05)*. ACM, New York, NY, USA, 87–96. doi:10.1145/1073368.1073380

Josh Barnes and Piet Hut. 1986. A Hierarchical $O(N \log N)$ Force-Calculation Algorithm. *Nature* 324, 6096 (1986), 446–449. doi:10.1038/324446a0

Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. 2022. Mip-NeRF 360: Unbounded Anti-Aliased Neural Radiance Fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 5470–5479.

J. Thomas Beale and Andrew Majda. 1985. High Order Accurate Vortex Methods with Explicit Velocity Kernels. *J. Comput. Phys.* 58, 2 (1985), 188–208. doi:10.1016/0021-9991(85)90176-7

Robert Bridson, Jim Hourihan, and Marcus Nordenstam. 2007. Curl-Noise for Procedural Fluid Flow. *ACM Transactions on Graphics* 26, 3 (2007), 46:1–46:3. doi:10.1145/1276377.1276435

Tyson Brochu, Todd Keeler, and Robert Bridson. 2012. Linear-Time Smoke Animation with Vortex Sheet Meshes. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '12)*. ACM, New York, NY, USA, 87–95.

Alexandre Joel Chorin. 1973. Numerical Study of Slightly Viscous Flow. *Journal of Fluid Mechanics* 57, 4 (1973), 785–796. doi:10.1017/S0022112073002016

Mengyu Chu, Lingjie Liu, Quan Zheng, Erik Franz, Hans-Peter Seidel, Christian Theobalt, and Rhaleb Zayer. 2022. Physics Informed Neural Fields for Smoke Reconstruction with Sparse Data. *ACM Transactions on Graphics* 41, 4 (2022), 119:1–119:14. doi:10.1145/3528223.3530169

Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrian Jarabo. 2025. Don't Splat Your Gaussians: Volumetric Ray-Traced Primitives for Modeling and Rendering Scattering and Emissive Media. *ACM Transactions on Graphics* 44, 1 (2025), 10:1–10:17. doi:10.1145/3711853

Georges-Henri Cottet and Petros D. Koumoutsakos. 2000. *Vortex Methods: Theory and Practice*. Cambridge University Press.

Keenan Crane, Ignacio Llamas, and Sarah Tariq. 2007. Real-Time Simulation and Rendering of 3D Fluids. In *GPU Gems 3*, Hubert Nguyen (Ed.). Addison-Wesley, Chapter 30, 633–675.

Philippe Decaudin and Fabrice Neyret. 2009. Volumetric Billboards. *Computer Graphics Forum* 28, 8 (2009), 2079–2089. doi:10.1111/j.1467-8659.2009.01354.x

Pierre Degond and Sylvie Mas-Gallic. 1989. The Weighted Particle Method for Convection-Diffusion Equations. Part 1: The Case of an Isotropic Viscosity. *Math. Comp.* 53, 188 (1989), 485–507. doi:10.2307/2008716

Ronald Fedkiw, Jos Stam, and Henrik Wann Jensen. 2001. Visual Simulation of Smoke. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '01)*. ACM, New York, NY, USA, 15–22. doi:10.1145/383259.383260

Yutao Feng, Xiang Feng, Yintong Shang, Ying Jiang, Chang Yu, Zeshun Zong, Tianjia Shao, Hongzhi Wu, Kun Zhou, Chenfanfu Jiang, and Yin Yang. 2025. Gaussian Splashing: Unified Particles for Versatile Motion Synthesis and Rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 518–529.

Manuel Noronha Gamito, Pedro Faria Lopes, and Mário Rui Gomes. 1995. Two-dimensional Simulation of Gaseous Phenomena Using Vortex Particles. In *Computer Animation and Simulation '95*. Springer Vienna, 3–15. doi:10.1007/978-3-7091-9435-5_1

Yue Gao, Hong-Xing Yu, Bo Zhu, and Jiajun Wu. 2025. FluidNexus: 3D Fluid Reconstruction and Prediction from a Single Video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 26091–26101.

Claude Greengard. 1985. The Core Spreading Vortex Method Approximates the Wrong Equation. *J. Comput. Phys.* 61, 2 (1985), 345–348. doi:10.1016/0021-9991(85)90091-9

Leslie Greengard and Vladimir Rokhlin. 1987. A Fast Algorithm for Particle Simulations. *J. Comput. Phys.* 73, 2 (1987), 325–348. doi:10.1016/0021-9991(87)90140-9

Torsten Hädrich, Daniel T. Banuti, Wojtek Palubicki, Sören Pirk, and Dominik L. Michels. 2021. Fire in Paradise: Mesoscale Simulation of Wildfires. *ACM Transactions on Graphics* 40, 4 (2021), 163:1–163:15. doi:10.1145/3450626.3459954

Mark J. Harris. 2004. Fast Fluid Dynamics Simulation on the GPU. In *GPU Gems*, Randima Fernando (Ed.). Addison-Wesley, Chapter 38, 637–665.

Torsten Hoefler and Roberto Belli. 2015. Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, Article 73, 12 pages. doi:10.1145/2807591.2807644

Christopher Horvath and Willi Geiger. 2009. Directable, High-Resolution Simulation of Fire on the GPU. *ACM Transactions on Graphics* 28, 3 (2009), 41:1–41:8. doi:10.1145/1531326.1531347

Krishna Murthy Jatavallabhula, Miles Macklin, Florian Golemo, Vikram Voleti, Linda Petrini, Martin Weiss, Breandan Considine, Jerome Parent-Lévesque, Kevin Xie, Kenny Erleben, Liam Paull, Florian Shkurti, Derek Nowrouzezahrai, and Sanja Fidler. 2021. gradSim: Differentiable Simulation for System Identification and Visuomotor Control. In *International Conference on Learning Representations (ICLR)*.

Ying Jiang, Chang Yu, Tianyi Xie, Xuan Li, Yutao Feng, Huamin Wang, Minchen Li, Henry Lau, Feng Gao, Yin Yang, and Chenfanfu Jiang. 2024. VR-GS: A Physical Dynamics-Aware Interactive Gaussian Splatting System in Virtual Reality. In *ACM SIGGRAPH 2024 Conference Papers (SIGGRAPH '24)*. ACM, New York, NY, USA, Article 78, 1 pages. doi:10.1145/3641519.3657448

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* 42, 4 (2023), 139:1–139:14. doi:10.1145/3592433

Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. 2024. A Hierarchical 3D Gaussian Representation for Real-Time Rendering of Very Large Datasets. *ACM Transactions on Graphics* 43, 4 (2024), 62:1–62:15. doi:10.1145/3658160

Theodore Kim, Nils Thürey, Doug James, and Markus Gross. 2008. Wavelet Turbulence for Fluid Simulation. *ACM Transactions on Graphics* 27, 3 (2008), 50:1–50:6. doi:10.1145/1360612.1360649

Horace Lamb. 1932. *Hydrodynamics* (6th ed.). Cambridge University Press.

Anthony Leonard. 1980. Vortex Methods for Flow Simulation. *J. Comput. Phys.* 37, 3 (1980), 289–335. doi:10.1016/0021-9991(80)90040-6

Yuchen Lin, Chenguo Lin, Jianjin Xu, and Yadong Mu. 2025. OmniPhysGS: 3D Constitutive Gaussians for General Physics-Based Dynamics Generation. In *International Conference on Learning Representations (ICLR)*.

Keith Lindsay and Robert Krasny. 2001. A Particle Method and Adaptive Treecode for Vortex Sheet Motion in Three-Dimensional Flow. *J. Comput. Phys.* 172, 2 (2001), 879–907. doi:10.1006/jcph.2001.6748

Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. 2024. Dynamic 3D Gaussians: Tracking by Persistent Dynamic View Synthesis. In *International Conference on 3D Vision (3DV)*.

Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T. Barron, and Yinda Zhang. 2025. EVER: Exact Volumetric Ellipsoid Rendering for Real-Time View Synthesis. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 4930–4939.

Nicolas Moenne-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 2024. 3D Gaussian Ray Tracing: Fast Tracing of Particle Scenes. *ACM Transactions on Graphics* 43, 6 (2024), 1–19. doi:10.1145/3687934

Joseph J. Monaghan. 1992. Smoothed Particle Hydrodynamics. *Annual Review of Astronomy and Astrophysics* 30 (1992), 543–574. doi:10.1146/annurev.aa.30.090192.

002551
Krzysztof Narkowicz. 2016. ACES Filmic Tone Mapping Curve. https://knarkowicz.wordpress.com/2016/01/06/aces-filmic-tone-mapping-curve/. Blog post.
Jakob Nazarenus, Simin Kou, Fang-Lue Zhang, Wojtek Palubicki, Dominik L. Michels, Sören Pirk, and Reinhard Koch. 2025. Gaussians on Fire: High-Frequency Reconstruction of Flames. arXiv:2511.22459.
Duc Quang Nguyen, Ronald Fedkiw, and Henrik Wann Jensen. 2002. Physically Based Modeling and Animation of Fire. In *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '02)*. ACM, New York, NY, USA, 721–728. doi:10.1145/566570.566643
Sang Il Park and Myoung Jun Kim. 2005. Vortex Fluid for Gaseous Phenomena. In *Proceedings of the 2005 ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA '05)*. ACM, New York, NY, USA, 261–270. doi:10.1145/1073368.1073406
Vincent Pegoraro and Steven G. Parker. 2006. Physically-Based Realistic Fire Rendering. In *Proceedings of the Second Eurographics Workshop on Natural Phenomena (NPH '06)*. 51–59. doi:10.2312/NPH/NPH06/051-059
Tobias Pfaff, Nils Thuerey, Jonathan Cohen, Sarah Tariq, and Markus Gross. 2010. Scalable Fluid Simulation Using Anisotropic Turbulence Particles. *ACM Transactions on Graphics* 29, 6 (2010), 174:1–174:8. doi:10.1145/1882261.1866196
Tobias Pfaff, Nils Thuerey, and Markus Gross. 2012. Lagrangian Vortex Sheets for Animating Fluids. *ACM Transactions on Graphics* 31, 4 (2012), 112:1–112:8. doi:10.1145/2185520.2185608
Sören Pirk, Michał Jarząbek, Torsten Hädrich, Dominik L. Michels, and Wojciech Palubicki. 2017. Interactive Wood Combustion for Botanical Tree Models. *ACM Transactions on Graphics* 36, 6 (2017), 197:1–197:12. doi:10.1145/3130800.3130814
Louis F. Rossi. 1996. Resurrecting Core Spreading Vortex Methods: A New Scheme That Is Both Deterministic and Convergent. *SIAM Journal on Scientific Computing* 17, 2 (1996), 370–397. doi:10.1137/S1064827593254397
Philip G. Saffman. 1992. *Vortex Dynamics*. Cambridge University Press.
Hagit Schechter and Robert Bridson. 2008. Evolving Sub-Grid Turbulence for Smoke Animation. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. Eurographics Association, 1–7. doi:10.2312/SCA/SCA08/001-007
Andrew Selle, Nick Rasmussen, and Ronald Fedkiw. 2005. A Vortex Particle Method for Smoke, Water and Explosions. *ACM Transactions on Graphics* 24, 3 (2005), 910–914. doi:10.1145/1073204.1073282
Qianfan Shen, Ningxiao Tao, Qiyu Dai, Tianle Chen, Minghan Qin, Yongjie Zhang, Mengyu Chu, Wenzheng Chen, and Baoquan Chen. 2026. FieryGS: In-the-Wild Fire Synthesis with Physics-Integrated Gaussian Splatting. In *International Conference on Learning Representations (ICLR)*.
Yunhao Shui, Fuhong Zhang, Can Gao, Hao Xue, Zhiyin Ma, Gang Xun, and Xuesong Li. 2024. FlameGS: Reconstruct Flame Light Field via Gaussian Splatting. In *Proc. SPIE 13652, International Conference on Signal Processing and Neural Network Applications (SPNNA 2024)*. doi:10.1117/12.3065237
Jos Stam. 1999. Stable Fluids. In *Proceedings of the 26th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '99)*. ACM, New York, NY, USA, 121–128. doi:10.1145/311535.311548
John Steinhoff and David Underhill. 1994. Modification of the Euler Equations for "Vorticity Confinement": Application to the Computation of Interacting Vortex Rings. *Physics of Fluids* 6, 8 (1994), 2738–2744. doi:10.1063/1.868164
Tamás Umenhoffer, László Szirmay-Kalos, and Gábor Szijártó. 2006. Spherical Billboards and Their Application to Rendering Explosions. In *Proceedings of Graphics Interface 2006*. 57–63.
Sinan Wang, Yitong Deng, Molin Deng, Hong-Xing Yu, Junwei Zhou, Duowen Chen, Taku Komura, Jiajun Wu, and Bo Zhu. 2024. An Eulerian Vortex Method on Flow Maps. *ACM Transactions on Graphics* 43, 6 (2024), 248:1–248:14. doi:10.1145/3687996
Sinan Wang, Junwei Zhou, Fan Feng, Zhiqi Li, Yuchen Sun, Duowen Chen, Greg Turk, and Bo Zhu. 2025. Fluid Simulation on Vortex Particle Flow Maps. *ACM Transactions on Graphics* 44, 4 (2025), 1–24. doi:10.1145/3731198
Steffen Weißmann and Ulrich Pinkall. 2009. Real-time Interactive Simulation of Smoke Using Discrete Integrable Vortex Filaments. In *Workshop on Virtual Reality Interactions and Physical Simulations (VRIPHYS)*. Eurographics Association, 1–10. doi:10.2312/PE/vriphys/vriphys09/001-010
Steffen Weißmann and Ulrich Pinkall. 2010. Filament-Based Smoke with Vortex Shedding and Variational Reconnection. *ACM Transactions on Graphics* 29, 4 (2010), 115:1–115:12. doi:10.1145/1778765.1778852
Grégoire S. Winckelmans and Anthony Leonard. 1993. Contributions to Vortex Particle Methods for the Computation of Three-Dimensional Incompressible Unsteady Flows. *J. Comput. Phys.* 109, 2 (1993), 247–273. doi:10.1006/jcph.1993.1216
Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 2024. 4D Gaussian Splatting for Real-Time Dynamic Scene Rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 20310–20320.
Tianyi Xie, Zeshun Zong, Yuxing Qiu, Xuan Li, Yutao Feng, Yin Yang, and Chenfanfu Jiang. 2024. PhysGaussian: Physics-Integrated 3D Gaussians for Generative Dynamics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 4389–4398.
Jingrui Xing, Bin Wang, Mengyu Chu, and Baoquan Chen. 2025. Gaussian Fluids: A Grid-Free Fluid Solver Based on Gaussian Spatial Representation. In *ACM SIGGRAPH 2025 Conference Papers*. ACM, New York, NY, USA. doi:10.1145/3721238.3730620
Larry Yaeger, Craig Upson, and Robert Myers. 1986. Combining Physical and Visual Simulation—Creation of the Planet Jupiter for the Film "2010". In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*. ACM, 85–93. doi:10.1145/15922.15895
Hong-Xing Yu, Yang Zheng, Yuan Gao, Yitong Deng, Bo Zhu, and Jiajun Wu. 2023. Inferring Hybrid Neural Fluid Fields from Videos. In *Advances in Neural Information Processing Systems (NeurIPS)*.
Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. 2024. Mip-Splatting: Alias-Free 3D Gaussian Splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 19447–19456.
Tianyuan Zhang, Hong-Xing Yu, Rundi Wu, Brandon Y. Feng, Changxi Zheng, Noah Snavely, Jiajun Wu, and William T. Freeman. 2024. PhysDreamer: Physics-Based Interaction with 3D Objects via Video Generation. In *Computer Vision – ECCV 2024*. Springer, 388–406. doi:10.1007/978-3-031-72627-9_22
Wenran Zhang, Yuxiang Cai, Letian Huang, Dongwei Ye, Jie Guo, and Bo Ren. 2026. GauSmoke: Hybrid Physics-Optical Gaussian Splatting for Sparse Smoke Reconstruction. In *ACM SIGGRAPH 2026 Conference Papers*. ACM, 1–11. doi:10.1145/3799902.3811148
Xinxin Zhang and Robert Bridson. 2014. A PPPM Fast Summation Method for Fluids and Beyond. *ACM Transactions on Graphics* 33, 6 (2014), 206:1–206:11. doi:10.1145/2661229.2661261
Zhiwei Zhao, Alan Zhao, Minchen Li, and Yixin Hu. 2025. Vid2Fluid: 3D Dynamic Fluid Assets from Single-View Videos with Generative Gaussian Splatting. arXiv:2503.00868.
Licheng Zhong, Hong-Xing Yu, Jiajun Wu, and Yunzhu Li. 2024. Reconstruction and Simulation of Elastic Objects with Spring-Mass 3D Gaussians. In *European Conference on Computer Vision (ECCV)*.
Yongning Zhu and Robert Bridson. 2005. Animating Sand as a Fluid. *ACM Transactions on Graphics* 24, 3 (2005), 965–972. doi:10.1145/1073204.1073298
Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross. 2001. EWA Volume Splatting. In *Proceedings of IEEE Visualization (VIS '01)*. 29–36. doi:10.1109/VISUAL.2001.964490

A Per-Slice Solver Pseudocode

```
step_all(dt):                          # once per frame
  for each sim layer L:                # Phase A: emit + pack + build
      emit at source (budgeted)
      for particle i in L:             # dynamics temperature
          if in source band: T_i += dt*s0*exp(-ks*x~^2)
          T_i *= (1 - 0.5*kc*dt); clamp
          if nu > 0: sig_i^2 += 2*nu*dt   # core spreading
      Morton-sort particles; pack SoA in sorted order
      build quadtree; per node: signed Gamma, |Gamma|-centroid,
          signed dipole, matched sig_c^2, MAC radius
  director = merge top tree levels across layers
           + ground-mirror images (x,-y,-Gamma)
  if kz > 0: pairwise exchange between adjacent layers:
      d = dt*kz*eta_ij*(G_j - G_i); G_i += d; G_j -= d

step_slice(L, dt):                     # Phase C: N-body + update
  for particle i:
      traverse tree once, accumulating in closed form:
        u_i, grad-u L_i          # Lamb-Oseen kernel + analytic gradient
          S=sum w, sum w*T, moment-renormalized dT/dx   # mixture T
      T~ = (sum w*T)/S
      dTdx = sum((T_j-T_i)(x_j-x_i)w) / sum((x_j-x_i)^2 w)
      v = u_i + beta_b*T~ * y-hat    # direct buoyancy
      v += c*(U_director(x_i) - v)  # nudge (Eq. couple)
      v += turb*curl_noise(x_i,t) + wind; clamp |v|
  for particle i:                  # integrate (frame-lagged baroclinic)
      G_i = (1-lam*dt)*G_i + dt*beta*dTdx_i; clamp
      Sigma_i evolved by L_i         # rendered covariance (Sec. 5.2)
      x_i += dt*v_i
      Tc_i -= dt*kappa*Tc_i^2        # color temperature (render)
      d_i *= (1 - 0.3*dt)
      recycle if cold/exhausted/out of bounds
```

B Supplemental Material

Video (85 s). Seven single-take sequences, unedited, ordered results first. No sequence in it uses the captured reconstruction: the

loaded scene throughout is the synthetic test one (§5.5); the captured scene is covered by the three separate clips described next. It opens on three shots of what the system produces—a flame in close-up, the same solver burning inside that loaded scene under a slow parallax arc, and one particle state re-shaded from fire to smoke—then turns to evidence. The next three are *side-by-side solver comparisons*: the retained vortex-in-cell baseline on the left and the grid-free solver on the right, run from the *same seed* at the *same fixed timestep* through the identical renderer and shading, so every visual difference is attributable to the solver—(1) fixed camera, (2) a constant-rate orbit, and (3) interaction (scripted source drag, wind, and the fire/smoke re-shading of one particle state, the same scripted inputs applied to both sides). The finale shows the fire-spread loop on that same synthetic scene, in which burning splats feed the solver and the plume above the fire is sustained by the scene; the loop on the captured reconstructions is reported as stills and timings only (§5.7), since at 380k splats and above it steps several times slower than real time. All sequences run the shipping configuration (N=9 × 600). Frames are rendered deterministically at $\Delta t = 1/60$ s from per-sequence seeds and encoded at 60 fps, so playback speed equals simulation speed; on the three side-by-side comparison sequences each side's frame-time overlay shows the rate measured for *that* solver on the reference machine (15.5 ms/65 FPS grid, 17.1 ms/58 FPS grid-free), consistent with Table 4. Those overlay rates are the fire alone. They do not apply to the two sequences shot inside a loaded scene, which additionally pay that scene's draw: end to end those run at 36.4 ms (28 FPS) at the same shipping configuration (§7). Every sequence in this video predates the captured reconstruction and uses the synthetic test scene; none of them shows the campsite capture of §5.5, whose composited frame time is 160.9 ms, nor KITCHEN, whose composited frame time is 292.4 ms. The interactive analogue of the comparison sequences ships with the demo: a lockstep side-by-side page driving two instances from one clock, seed and control stream.

Captured-scene clips (26 s, three files). Three further sequences ship as separate files alongside that video, and are the ones that put the solver inside the *captured* reconstruction: KITCHEN at 729 826 Gaussians and degree-3 spherical harmonics, at the training-view-derived camera of Figure 1 and Figure 10. (1) *Fixed camera*, 10 s at 1920×1080: the flame burns on the table behind the model loader, whose lift arms and black hydraulic linkage occlude it per primitive throughout. (2) *Parallax*, 5 s: the azimuth sweeps 0.416 → 0.316 → 0.416 rad, so the occlusion is shown to hold across a moving viewpoint rather than at one chosen frame—the shot this submission previously named as the one it would add first. (3) *Reveal*, 11 s at 1920×1080: three seconds of the reconstruction on its own—180 bit-identical frames, none of our primitives drawn—and then eight seconds of ignition from the same seed the figures use. All three are deterministic per-frame exports at $\Delta t = 1/60$ s encoded at 60 fps, so playback speed again equals simulation speed; none of them is a screen recording, since a composited frame at this splat count costs 292.4 ms (§7). The simulation is the real-time one—what a three-quarter-million-splat reconstruction adds is rasterization, not solver work (§7). The parallax clip is rendered at 1544×1080 and pillarboxed for delivery rather than shot at 16:9. The demo's field of view is vertical, so 16:9 is the solved framing plus 188 px of width on each side; at the fixed camera those strips are ordinary room content, but at a yaw offset of 0.10 rad the added left strip is 45% empty. The capture has no geometry that far off-axis, and widening the frame only shows that fact.

Code. The complete system is the single dependency-free HTML file released with this submission—including the retained vortex-in-cell baseline (with the runtime-selectable 64×96 control resolution), the exact-Biot–Savart reference path, and the in-browser benchmark/treecode-error harness—together with the 3D reference solver, the spectral-analysis scripts (single- and multi-seed, including the grid-refinement, core-radius and timestep controls of §7), and the scene generator used for the composition results.